

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Nhật Phong

**TỐI ƯU HÓA CÂN BẰNG TRONG TRÒ CHƠI
ROGUELIKE DECKBUILDER BẰNG THUẬT
TOÁN
TIẾN HÓA ĐA MỤC TIÊU VÀ HỌC TĂNG
CƯỜNG**

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Khoa học máy tính

Cán bộ hướng dẫn: PGS. TS. Nguyễn Trí Thành

(ký tên)

HÀ NỘI – 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Nguyễn Nhật Phong

**OPTIMIZING GAME BALANCE IN
ROGUELIKE
DECKBUILDERS VIA MULTI-OBJECTIVE
EVOLUTIONARY ALGORITHMS AND
REINFORCEMENT LEARNING**

BACHELOR'S THESIS

Major: Computer Science

Supervisor: Assoc. Prof. Dr. Nguyễn Trí Thành

(signature)

HANOI – 2026

TÓM TẮT

Cân bằng trò chơi là một trong những thách thức quan trọng và tốn nhiều công sức nhất trong lĩnh vực phát triển game, đặc biệt đối với thể loại Roguelike Deckbuilder – nơi hàng trăm tham số liên kết với nhau như sát thương thẻ bài, chỉ số kẻ thù, hiệu ứng cổ vật và phân bố bản đồ phải cùng tồn tại để tạo ra trải nghiệm người chơi công bằng và hấp dẫn. Phương pháp tinh chỉnh thủ công truyền thống tốn thời gian, dễ sai sót và không có khả năng khám phá không gian tham số rộng lớn một cách có hệ thống.

Luận văn này trình bày một hệ thống tự động cân bằng game hoàn chỉnh cho một prototype Roguelike Deckbuilder được xây dựng bằng C# (.NET 9). Hệ thống gồm ba tầng tích hợp chặt chẽ. Tầng Game Core hiện thực hóa toàn bộ logic trò chơi bao gồm chiến đấu theo lượt, bản đồ sinh ngẫu nhiên theo thủ tục, hệ thống xây dựng bộ bài với hơn 30 thẻ bài và 20 cổ vật, cùng nhiều loại phòng khác nhau. Một tác nhân heuristic tắt định (DHA) được phát triển trước tiên để xác nhận tính khả chơi của game và thiết lập các chỉ số cơ sở trước khi tiến hành huấn luyện tác nhân học tăng cường.

Đóng góp chính của luận văn là giới thiệu bốn tác nhân Học Tăng Cường được huấn luyện bằng thuật toán MaskablePPO, đại diện cho các lối chơi khác nhau: Tấn công, Phòng thủ, Cân bằng và Thích nghi. Mỗi tác nhân được định hình thông qua hàm phần thưởng tùy chỉnh và huấn luyện trong 8 triệu bước trên phần cứng GPU. Cơ chế che mặt nạ hành động không hợp lệ và kỹ thuật ngẫu nhiên hóa miền được áp dụng để nâng cao sự ổn định huấn luyện và khả năng tổng quát hóa của tác nhân. Cả bốn tác nhân đạt tỷ lệ thắng trong khoảng 24 đến 34%, là tiêu chuẩn thực tế phù hợp cho trò chơi có yếu tố ngẫu nhiên cao.

Tầng Tối ưu hóa triển khai hai phương pháp tiến hóa: Thuật toán Di truyền đơn mục tiêu hoạt động trên khoảng 500 tham số trực tiếp, và thuật toán NSGA-II với bộ gen phân cấp gồm khoảng 50 tham số tối ưu hóa đồng thời ba mục tiêu là Cân bằng, Thu hút và Tính nhất quán. Sau 30 thế hệ với quần thể 100 cá thể, NSGA-II đạt Điểm Cân bằng 0,960, Điểm Thu hút 0,900 và Tính nhất quán hoàn hảo 1,000, đồng thời tăng số thẻ bài có giá trị từ 12 lên 14 trên tổng số 30 thẻ, trong vòng 49 phút – nhanh hơn 33% so với GA đơn mục tiêu.

Nghiên cứu này chứng minh rằng sự kết hợp giữa biểu diễn bộ gen phân cấp, tối ưu hóa đa mục tiêu và các tác nhân học tăng cường được huấn luyện tạo ra một framework cân bằng game tự động mạnh mẽ, có thể mở rộng, vượt trội đáng kể so với cả phương pháp điều chỉnh thủ công lẫn các phương pháp di truyền đơn mục tiêu truyền thống.

Từ khóa: Cân bằng trò chơi, *Roguelike Deckbuilder*, *NSGA-II*, Học tăng cường.

ABSTRACT

Game balancing is one of the most critical and labor-intensive challenges in game development, particularly for Roguelike Deckbuilder games, where hundreds of interconnected parameters – card damages, enemy statistics, relic effects, and map distributions – must coexist to create a fair and engaging player experience. Traditional manual tuning is slow, error-prone, and incapable of exploring the vast parameter space systematically.

This thesis presents a complete automated game-balancing framework for a custom Roguelike Deckbuilder prototype built in C# (.NET 9). The framework is composed of three tightly integrated layers. The **Core Game Layer** implements the full game logic including turn-based combat, a procedurally generated map, a deckbuilding system with over 30 cards and 20 relics, and multiple room types. A rule-based **Deterministic Heuristic Agent** was first developed to validate game playability and establish baseline metrics.

The key contribution of this thesis is the introduction of four **Reinforcement Learning (RL) Agents** trained via **Maskable Proximal Policy Optimization (MaskablePPO)** to replace the deterministic agent. These agents embody distinct play styles – *Aggressive*, *Defensive*, *Balanced*, and *Adaptive* – each shaped through custom reward functions and trained over five million timesteps on GPU hardware. Invalid Action Masking and Domain Randomization are employed to improve training stability and agent generalization. All four agents converge to a win rate of approximately 24–34%, a realistic benchmark for a game with significant stochastic elements.

The **Optimization Layer** employs two evolutionary approaches: a **Single-Objective Genetic Algorithm (GA)** operating on ~ 500 direct parameters, and **NSGA-II** with a hierarchical genome of ~ 50 parameters optimizing three objectives simultaneously – *Balance*, *Engagement*, and *Coherence*. The RL agents serve as the primary evaluators within the optimization loop. Across 30 generations with a population of 100, NSGA-II achieves a Balance Score of 0.960, Engagement Score of 0.900, and perfect Coherence (1.000) while eliminating all trap cards and increasing the number of viable cards from 12 to 14, all within 49 minutes – 33% faster than the single-objective GA.

This research demonstrates that combining hierarchical genome representation with multi-objective optimization and learned RL agents produces a robust, extensible framework for automatic game balancing that significantly outperforms both manual tuning and flat single-

objective genetic approaches.

Keywords: *Roguelike Deckbuilder, NSGA-II, Reinforcement Learning, Game Balance Optimization.*

DECLARATION OF AUTHORSHIP

I hereby declare that this graduation thesis is my own original work. All results, analyses, and conclusions presented in this document are truthful and have not been copied from any other source in any form whatsoever. All references to external sources have been properly cited and acknowledged in accordance with academic regulations.

If any form of misconduct is discovered, I accept full responsibility for the content of this thesis. The University of Engineering and Technology bears no liability for any copyright or intellectual property violations I may have committed during the preparation of this work.

Hanoi, May 2026

Author

Nguyễn Nhật Phong

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, **Assoc. Prof. Dr. Nguyễn Trí Thành**, for his invaluable guidance, insightful feedback, and continuous encouragement throughout the entire process of completing this thesis. His expertise in algorithms and artificial intelligence has shaped both the direction and quality of this research.

I am also grateful to the faculty and staff of the **Faculty of Information Technology, University of Engineering and Technology, Vietnam National University, Hanoi**, for providing me with a solid academic foundation over four years of study. The knowledge and skills I have gained here have made this project possible.

Special thanks go to my fellow students and friends who provided moral support, engaged in stimulating technical discussions, and helped test early versions of the game system. Their enthusiasm and feedback were a constant source of motivation.

Finally, I owe immeasurable gratitude to my family, whose unwavering love, patience, and encouragement have sustained me through every challenge. This work is dedicated to them.

Hanoi, May 2026

Nguyễn Nhật Phong

Contents

Tóm tắt	i
Abstract	iii
Declaration of Authorship	v
Acknowledgements	vi
Table of Contents	vii
List of Figures	xi
List of Tables	xiii
List of Abbreviations	xiv
List of Abbreviations	xiv
Introduction	1
1 BACKGROUND AND RELATED WORK	5
1.1 Roguelike Deckbuilder Games	5
1.1.1 Roguelike Games	5
1.1.2 Deckbuilding Mechanics	5
1.1.3 Roguelike Deckbuilders	5
1.1.4 Balancing Challenges in Roguelike Deckbuilders	6
1.2 Genetic Algorithms	6
1.2.1 Biological Motivation	6
1.2.2 Core Components	7
1.3 NSGA-II: Multi-Objective Evolutionary Optimization	8
1.3.1 Multi-Objective Problem Formulation	8
1.3.2 The NSGA-II Algorithm	8
1.4 Reinforcement Learning	9

1.4.1	Markov Decision Processes	9
1.4.2	Deep Reinforcement Learning	10
1.4.3	Proximal Policy Optimization (PPO)	10
1.4.4	Invalid Action Masking	10
1.4.5	Domain Randomization	11
1.5	Related Work	11
1.5.1	Automated Game Balancing	11
1.5.2	RL for Card Games	12
1.5.3	Procedural Content Generation	12
2	SYSTEM ANALYSIS AND DESIGN	13
2.1	Requirements Analysis	13
2.1.1	Functional Requirements	13
2.1.2	Non-Functional Requirements	13
2.2	System Architecture	14
2.3	Core Game Engine Design	14
2.3.1	Game Structure Overview	14
2.3.2	Combat System Design	15
2.4	Deterministic Heuristic Agent Design	17
2.4.1	Purpose and Role	17
2.4.2	Decision Logic	17
2.5	Reinforcement Learning Environment Design	18
2.5.1	Observation Space	18
2.5.2	Action Space	19
2.5.3	Episode Structure	20
2.6	RL Agent Persona Design	20
2.6.1	Design Rationale	20
2.6.2	Aggressive Agent	20
2.6.3	Defensive Agent	21
2.6.4	Balanced Agent	21
2.6.5	Adaptive Agent	21
2.7	Optimization Layer Design	22
2.7.1	Data Flow	22
2.7.2	Genome Representations	23
2.7.3	Fitness Functions	24
3	IMPLEMENTATION	25

3.1	Development Environment	25
3.2	Core Game Engine Implementation	25
3.2.1	Project Structure	25
3.2.2	Game Controller	26
3.2.3	Combat System	27
3.2.4	Map Generation	27
3.2.5	Data-Driven Design	28
3.2.6	Heuristic AI Implementation	28
3.3	Cross-Platform C# – Python Integration	29
3.3.1	Motivation and Architecture	29
3.3.2	Kaggle Setup Script	29
3.4	Reinforcement Learning Environment	30
3.4.1	DotNetGameEnv (Gymnasium Wrapper)	30
3.4.2	Domain Randomization	31
3.4.3	Observation Construction	32
3.5	Agent Training Pipeline	33
3.5.1	Two-Phase Training Strategy	33
3.5.2	MaskablePPO Hyperparameters	34
3.5.3	Deck Quality Reward	34
3.5.4	Training Infrastructure	34
3.6	Evolutionary Optimization Implementation	36
3.6.1	Single-Objective Genetic Algorithm	36
3.6.2	NSGA-II Multi-Objective Optimizer	36
3.6.3	Python-Side Fitness Evaluation for RL Agents	37
4	EXPERIMENTS AND EVALUATION	39
4.1	Experimental Setup	39
4.1.1	Hardware Configuration	39
4.1.2	Optimization Algorithm Parameters	40
4.1.3	Target Metrics	40
4.2	RL Agent Training Results	40
4.2.1	Final Win Rates	40
4.2.2	Behavioral Differentiation	41
4.2.3	Training Convergence	42
4.3	Game Optimization Results	42
4.3.1	Single-Objective GA Results	42
4.3.2	NSGA-II Multi-Objective Results	43

4.3.3	Pareto Front Analysis	45
4.3.4	Comparative Summary	46
4.4	Card Viability Analysis	47
4.5	Comparison with Heuristic Agent Baseline	49
4.6	Discussion	49
4.6.1	Why NSGA-II Outperforms GA	49
4.6.2	Limitations of Current Evaluation	50
5	CONCLUSION	51
5.1	Summary of Achievements	51
5.2	Scientific and Practical Contributions	52
5.2.1	Scientific Contributions	52
5.2.2	Practical Contributions	52
5.3	Limitations	53
5.3.1	Agent Representativeness	53
5.3.2	Fixed Hierarchy Structure	53
5.3.3	Content Scope	54
5.3.4	Single-Character Evaluation	54
5.4	Future Work	54
5.4.1	RL-Integrated Optimization	54
5.4.2	NSGA-III and Many-Objective Extensions	54
5.4.3	Meta-Learning for Hierarchy Discovery	55
5.4.4	Human Playtest Integration	55
5.4.5	Expanded Game Content	55
5.4.6	Designer Tool	55
	References	56
	APPENDIX	58
5.5	Game Data JSON Structure	58
5.5.1	Card Data	58
5.5.2	Enemy Data	59
5.5.3	Relic Data	60
5.6	Algorithm Pseudocode	61
5.6.1	Single-Objective Genetic Algorithm	61
5.6.2	NSGA-II Multi-Objective Optimizer	62
5.7	Reward Function Summary	62
5.8	Observation Space Index Table	64

List of Figures

2.1	High-level system architecture showing data flow between the Optimization Layer, Simulation/Training Layer, Data Layer, and Core Game Layer.	14
2.2	Class diagram of the core game module showing the relationships between GameController, CombatManager, DeckManager, the AI interface, and the optimization layer.	16
2.3	Sequence diagram of one combat turn: the Combat Manager draws cards from the Deck Manager, queries the AI Agent for a decision, resolves the card action, executes the enemy turn, and checks win/loss conditions.	17
2.4	Data flow in the optimization loop. A genome is created by the optimization layer, applied to a scaled GameData copy, passed to the simulation runner which uses the AI agent to play games, and resulting statistics are evaluated into fitness scores that feed back to the optimizer.	22
4.1	GA fitness evolution over 30 generations. Overall fitness improves from 1.45 to 32.66 (+2,150%), with rapid initial gains levelling off by generation 10.	42
4.2	Evolution of Balance, Engagement, and Coherence scores for the best individual across GA generations. Balance improves most rapidly; Engagement stagnates.	43
4.3	NSGA-II hypervolume indicator across 30 generations. After a sharp initial rise in generations 1–3, the Pareto front stabilizes and continues to improve gradually.	44
4.4	Best Balance, Engagement, and Coherence scores across NSGA-II generations for the compromise solution on the Pareto front.	44
4.5	Three-dimensional Pareto front in (Balance, Engagement, Coherence) space. Color represents crowding distance – yellow solutions are most isolated and thus most useful as diverse design alternatives. Twelve non-dominated solutions were identified.	45
4.6	Two-dimensional projections of the NSGA-II Pareto front. Each point represents one non-dominated solution.	46

4.7	Card pick rate versus win rate when picked, colored by card type (blue: Attack, teal: Skill/Power). Ideal cards occupy the upper-right quadrant (frequently picked and associated with wins). Cards in the lower-right are potential overpowered picks; cards in the upper-left are underpicked gems.	48
4.8	Viable card count (green, top line) and trap card count (pink, bottom line) across generations. The viable card count stabilizes near 12–14; trap cards are eliminated by generation 8.	48

List of Tables

2.1	Observation space components (total: 148 dimensions).	19
3.1	Development environment specifications.	25
3.2	MaskablePPO hyperparameters for all four agents.	34
4.1	Hardware configuration used in experiments.	39
4.2	Comparison of GA and NSGA-II algorithm configurations.	40
4.3	Final win rates of the four RL agents evaluated over 200 deterministic episodes.	41
4.4	Comprehensive comparison of GA and NSGA-II performance after 30 generations.	47
4.5	Comparison of optimization outcomes using the Deterministic Heuristic Agent (DHA) versus RL agents as evaluators in NSGA-II (30 generations, 100 individuals).	49
5.1	Complete reward function summary for all four RL agents.	63
5.2	148-dimensional observation vector layout. All values are normalized to $[0, 1]$	64

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CLR	Common Language Runtime
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DLL	Dynamic Link Library
DQN	Deep Q-Network
DR	Domain Randomization
DRL	Deep Reinforcement Learning
GA	Genetic Algorithm
GPU	Graphics Processing Unit
HP	Hit Points (Health Points)
JSON	JavaScript Object Notation
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
MOEA	Multi-Objective Evolutionary Algorithm
NSGA-II	Non-dominated Sorting Genetic Algorithm II
PCG	Procedural Content Generation
PPO	Proximal Policy Optimization
RAM	Random Access Memory
RL	Reinforcement Learning
RNG	Random Number Generator
SBX	Simulated Binary Crossover
SDK	Software Development Kit
SB3	Stable-Baselines3
UET	University of Engineering and Technology
VNU	Vietnam National University

INTRODUCTION

1. Motivation

The video game industry has grown into one of the largest entertainment sectors in the world, generating over \$180 billion in annual revenue [1]. Within this landscape, the *Rogue-like Deckbuilder* sub-genre has seen remarkable popularity, with titles such as *Slay the Spire*, *Monster Train*, and *Inscryption* achieving both critical acclaim and millions of copies sold [2]. These games are characterized by procedurally generated maps, permadeath mechanics, and an evolving card collection that grows organically through each run.

Despite their commercial success, Roguelike Deckbuilders are notoriously difficult to balance. A well-balanced instance must satisfy several competing desiderata simultaneously: the game should be challenging enough to remain engaging but not so punishing that players abandon it in frustration; the card pool should offer multiple viable build paths so that strategic variety is preserved; and the difficulty curve should scale smoothly across the game’s fifteen or more floors so that no single floor constitutes an impassable wall. Achieving all of these properties simultaneously across the combinatorial space of possible card combinations, enemy encounters, relic synergies, and random deck draws is a task that defies purely manual tuning [3].

Traditional game-balancing practice relies on *playtesting*: human players repeatedly attempt the game, report friction points, and designers manually adjust parameters. While essential, this process is slow, expensive, and bounded by the cognitive bandwidth of the design team. It is also inherently subjective: what feels balanced to an expert speedrunner may seem brutally unfair to a casual player. Automated simulation offers a route to complementing playtesting with data-driven optimization, but it raises its own challenge: what agent should be used to simulate the player?

This thesis addresses both the balancing problem and the agent problem together. We design and train a suite of **Reinforcement Learning (RL)** agents that learn to play the game and subsequently serve as the evaluators inside an **evolutionary optimization loop** that searches for well-balanced game configurations.

2. Problem Statement

Let $\theta \in \mathbb{R}^n$ be a vector of game parameters (card damages, enemy health points, mana costs, room-type frequencies, etc.). Given a game simulator $G(\theta)$ and a population of player agents $\mathcal{A} = \{a_1, a_2, \dots, a_k\}$, we wish to find θ^* such that the resulting game satisfies:

1. **Balance:** The win rate $w(\theta, a)$ for a representative agent a lies within a target range $[w_{\min}, w_{\max}]$, and the game does not trivialize or become unwinnable for any agent persona.
2. **Engagement:** A large fraction of the available cards are “viable” – i.e., choosing them correlates positively with winning – supporting diverse build strategies.
3. **Coherence:** No card is a dominant trap – appearing attractive to the agent but consistently leading to defeat – and parameter scalars stay within an interpretable range.

Because these three objectives can conflict (making the game harder may increase balance but reduce engagement), we frame the search as a **multi-objective optimization problem** over the Pareto front of (Balance, Engagement, Coherence).

3. Research Objectives

The concrete objectives of this thesis are:

1. **Build a complete game prototype:** Implement a fully playable Roguelike Deckbuilder in C# with turn-based combat, a procedurally generated map, a rich card and relic pool, and a data-driven parameter system loadable from JSON.
2. **Establish a playability baseline:** Design and implement a rule-based *Deterministic Heuristic Agent (DHA)* that confirms the game is playable and provides early balancing data.
3. **Train four RL agent personas:** Replace the deterministic agent with four learned MaskablePPO agents representing distinct play styles (*Aggressive, Defensive, Balanced, Adaptive*), each trained with persona-specific reward shaping and Domain Randomization to generalize across varied game configurations.
4. **Implement and compare two evolutionary optimizers:** Deploy a *Single-Objective Genetic Algorithm (GA)* and the *NSGA-II* multi-objective algorithm; compare their effectiveness at discovering balanced game configurations.

5. **Analyze and visualize results:** Produce Pareto front analysis, card viability charts, and training curves to illuminate the behavior of both the optimization and the trained agents.

4. Scope and Methodology

The research scope covers the full software stack from game engine to optimization layer. The game prototype, while self-contained and playable, is designed as a research vehicle rather than a commercial product, and its parameter space is crafted to be representative of the real design challenge without requiring the scale of a production codebase.

Methodologically, the work proceeds through four phases:

1. **Foundation:** Literature review on Roguelike games, evolutionary algorithms, reinforcement learning, and related automated balancing research.
2. **System construction:** Implement the game engine (C# .NET 9), the cross-platform integration layer (pythonnet), the Gymnasium RL environment, and the optimization modules.
3. **Agent training:** Train the four MaskablePPO agents on Kaggle GPU infrastructure, iteratively refining reward functions and hyperparameters based on training logs.
4. **Optimization and evaluation:** Run GA and NSGA-II experiments; collect, analyze, and visualize results.

5. Main Contributions

The primary contributions of this thesis are:

- A **complete, open-source Roguelike Deckbuilder prototype** with a data-driven architecture that separates game logic from balance parameters, enabling fully automated parameter search.
- A **cross-platform integration bridge** between C# game logic and Python RL training, enabling high-throughput simulation on Linux Kaggle GPU kernels without network overhead.
- **Four trained RL personas** for game evaluation, each exhibiting measurably distinct behaviors (win-rate range 24–34%, statistically significant behavioral differences in HP expenditure and turn duration).

- An **empirical comparison of GA versus NSGA-II** for game balancing under identical simulation budgets, demonstrating that the hierarchical NSGA-II formulation converges 33% faster while achieving superior scores on all three objectives.
- A **reusable framework design** applicable to other card-based or turn-based games with minor adaptation.

6. Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 1 – Background and Related Work:** Reviews the Roguelike Deckbuilder genre, game balancing challenges, Genetic Algorithms, NSGA-II, Reinforcement Learning theory (MDP, PPO, invalid action masking), Domain Randomization, and related prior work.
- **Chapter 2 – System Analysis and Design:** Presents the three-tier architecture, the game engine design, the RL environment specification, the four agent persona designs, and the optimization layer design including genome representations and fitness functions.
- **Chapter 3 – Implementation:** Details the C# game engine implementation, cross-platform pythonnet integration, the Gymnasium environment wrapper, MaskablePPO training pipeline, and the GA/NSGA-II optimizer implementations.
- **Chapter 4 – Experiments and Evaluation:** Reports experimental results for both the RL training phase and the evolutionary optimization phase, including ablation analysis and comparative discussion.
- **Chapter 5 – Conclusion and Future Work:** Summarizes the achievements, discusses limitations, and outlines directions for future research.

CHAPTER 1

BACKGROUND AND RELATED WORK

1.1 Roguelike Deckbuilder Games

1.1.1 *Roguelike Games*

The Roguelike genre traces its origins to *Rogue* (1980), a terminal-based dungeon crawler distinguished by two defining features [4]. First, **procedural generation**: the dungeon layout, enemy placement, and item distribution are randomly generated anew for each attempt, ensuring that no two runs share the same experience. Second, **permadeath**: when the player character dies, all progress is lost and the player must begin a fresh run from the very start. Together, these mechanics create a gameplay loop that rewards learning and mastery over brute-force repetition.

Modern Roguelikes typically add turn-based gameplay, resource management (health, mana, gold), and a meta-progression system that unlocks new content across multiple runs. The genre rewards deep strategic thinking precisely because each decision can be decisive – there are no mid-run saves to fall back upon.

1.1.2 *Deckbuilding Mechanics*

The deckbuilder is a game category in which players begin with a small, weak deck of cards and progressively improve it by acquiring new cards and removing poor ones. Popularised by board games such as *Dominion* (2008) and *Thunderstone* (2009), the mechanic creates a layered strategic challenge: players must evaluate short-term combat utility against long-term deck synergy, deciding which cards to add based on an evolving game state. A well-constructed deck becomes exponentially more powerful than the sum of its parts through *synergies* – combinations of cards whose joint effect far exceeds their individual contributions.

1.1.3 *Roguelike Deckbuilders*

Merging the two genres, *Roguelike Deckbuilders* offer a run-based experience in which:

- Each run generates a unique map of rooms, enemies, and reward opportunities.

- After each combat, players select cards to add to their deck from a randomized reward pool.
- Death resets all progress, but accumulated strategic knowledge transfers.
- Victory requires both skillful in-combat card play and astute long-term deck construction.

Slay the Spire [2], released in 2019, is widely credited with establishing the genre’s commercial viability. The game’s deep card pool and enemy variety make it an ideal testbed for automated balancing research. This thesis builds a custom prototype inspired by *Slay the Spire*’s design principles.

1.1.4 Balancing Challenges in Roguelike Deckbuilders

Balancing a Roguelike Deckbuilder is substantially harder than balancing a deterministic game for several reasons:

1. **Combinatorial card interactions:** With n cards in the pool, there are 2^n possible subsets that could form a player’s deck, each with different synergies.
2. **Stochastic elements:** Random card draws, enemy intent patterns, and reward-pool composition introduce variance that makes it difficult to isolate balance problems from luck.
3. **Emergent strategies:** Players frequently discover synergies that designers did not anticipate, creating dominant strategies that must be patched post-release.
4. **Player skill heterogeneity:** A configuration balanced for expert players may be prohibitively difficult for novices, and vice versa.

1.2 Genetic Algorithms

1.2.1 Biological Motivation

Genetic Algorithms (GA) are a family of population-based metaheuristics that abstract the mechanisms of biological evolution [5]. A GA maintains a population of candidate solutions (genomes), evaluates their fitness with respect to an objective function, and iteratively applies selection, crossover, and mutation operators to produce progressively better populations. The key insight is that combining partial solutions from fit individuals (crossover) can discover configurations that neither parent achieves alone, while random perturbations (mutation) prevent premature convergence to local optima.

1.2.2 Core Components

Genome representation encodes a candidate solution as a chromosome. For continuous optimization problems, genomes are vectors of real numbers $\mathbf{g} = (g_1, g_2, \dots, g_n) \in \mathbb{R}^n$. In our game-balancing application, each gene g_i is a multiplier applied to a specific game parameter (e.g., the base damage of a particular card).

Fitness function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ assigns a scalar quality score to each genome. For single-objective GA:

$$F(\mathbf{g}) = w_1 \cdot f_{\text{balance}}(\mathbf{g}) + w_2 \cdot f_{\text{engagement}}(\mathbf{g}) + w_3 \cdot f_{\text{coherence}}(\mathbf{g}) \quad (1.1)$$

where w_1, w_2, w_3 are pre-determined weights.

Selection determines which genomes reproduce. *Tournament selection* of size k draws k individuals uniformly at random and returns the fittest, providing tunable selection pressure without requiring global fitness ranking.

Crossover combines two parent genomes to produce offspring. *Uniform crossover* assigns each gene to offspring 1 or offspring 2 with probability 0.5, effectively sampling a random hypercube between the two parents:

$$g_i^{\text{child}} = \begin{cases} g_i^{P_1} & \text{with probability 0.5} \\ g_i^{P_2} & \text{with probability 0.5} \end{cases} \quad (1.2)$$

Mutation perturbs genes stochastically to maintain diversity. Gaussian mutation adds zero-mean noise:

$$g'_i = g_i + \mathcal{N}(0, \sigma^2), \quad \text{with probability } p_m \quad (1.3)$$

Typical values are $p_m \in [0.01, 0.05]$ and $\sigma \in [0.05, 0.2]$ for normalized parameter ranges.

Elitism preserves a fraction of the top individuals unchanged across generations, ensuring that the best discovered solution never degrades.

1.3 NSGA-II: Multi-Objective Evolutionary Optimization

1.3.1 Multi-Objective Problem Formulation

Many real-world optimization problems require simultaneously maximizing several objectives that are in mutual tension [6]. The general multi-objective optimization problem is:

$$\text{maximize } \mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})) \quad (1.4)$$

Because improving one objective typically harms another, there is no single “optimal” solution. Instead, the goal is to characterize the *Pareto front* – the set of solutions that are non-dominated.

Definition 1.1 (Pareto Dominance). *Solution $\mathbf{x}_1$ **dominates** $\mathbf{x}_2$ (written $\mathbf{x}_1 \prec \mathbf{x}_2$) if and only if:*

$$\forall i : f_i(\mathbf{x}_1) \geq f_i(\mathbf{x}_2) \quad \text{and} \quad \exists j : f_j(\mathbf{x}_1) > f_j(\mathbf{x}_2) \quad (1.5)$$

Definition 1.2 (Pareto Optimal Set). *A solution $\mathbf{x}^*$ is **Pareto optimal** if no solution in the feasible space dominates it. The set of all Pareto optimal solutions forms the Pareto front.*

1.3.2 The NSGA-II Algorithm

NSGA-II (Non-dominated Sorting Genetic Algorithm II), proposed by Deb et al. [6], is the canonical multi-objective evolutionary algorithm. Its three key innovations over the original NSGA are: (1) efficient $\mathcal{O}(MN^2)$ fast non-dominated sorting, where M is the number of objectives and N is the population size; (2) a crowding distance metric for diversity preservation; and (3) an elitist selection strategy.

Fast non-dominated sorting partitions the population P into fronts $\mathcal{F}_1, \mathcal{F}_2, \dots$ where $\mathcal{F}_1$ contains all non-dominated individuals, $\mathcal{F}_2$ those dominated only by members of $\mathcal{F}_1$, and so on. The rank of individual i equals its front index.

Crowding distance d_i measures the density of solutions surrounding individual i in objective space:

$$d_i = \sum_{k=1}^m \frac{f_k^{(i+1)} - f_k^{(i-1)}}{f_k^{\max} - f_k^{\min}} \quad (1.6)$$

where $i \pm 1$ denotes the adjacent individuals when sorted by objective k . Boundary solutions receive $d_i = \infty$.

NSGA-II selection operator $\prec_n$ prefers lower rank, breaking ties by larger crowding

distance:

$$\mathbf{x}_1 \prec_n \mathbf{x}_2 \iff \text{rank}(\mathbf{x}_1) < \text{rank}(\mathbf{x}_2) \text{ or } (\text{rank}(\mathbf{x}_1) = \text{rank}(\mathbf{x}_2) \text{ and } d_1 > d_2) \quad (1.7)$$

This guarantees convergence toward the Pareto front while maintaining diversity along it.

Simulated Binary Crossover (SBX) and **Polynomial Mutation** are the preferred genetic operators for real-valued chromosomes in NSGA-II. SBX mimics single-point crossover for binary representations but operates directly on floating-point variables [6].

1.4 Reinforcement Learning

1.4.1 Markov Decision Processes

Reinforcement Learning (RL) is the branch of machine learning concerned with training an *agent* to make sequential decisions in an environment by maximizing a cumulative reward signal [7]. The formal framework is the **Markov Decision Process (MDP)**, defined as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$: state space – the set of all observable configurations of the environment.
- $\mathcal{A}$: action space – the set of possible agent actions.
- $\mathcal{P}(s' \mid s, a)$: transition dynamics – the probability of transitioning to state s' upon taking action a in state s .
- $\mathcal{R}(s, a, s')$: reward function – the immediate scalar signal received after each transition.
- $\gamma \in [0, 1]$: discount factor – controls the relative importance of future versus immediate rewards.

The agent's behavior is characterized by a **policy** $\pi(a \mid s)$, which is a probability distribution over actions conditioned on the current state. The objective is to find the optimal policy π^* that maximizes the **expected discounted return**:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (1.8)$$

1.4.2 Deep Reinforcement Learning

In environments with large or continuous state spaces (such as our game, with a 148-dimensional observation vector), the policy is parameterized by a deep neural network $\pi_\theta(a \mid s)$. The **action-value function** $Q^\pi(s, a)$ and **state-value function** $V^\pi(s)$ are defined as:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right] \quad (1.9)$$

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} [Q^\pi(s, a)] \quad (1.10)$$

The **advantage function** $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ measures how much better action a is compared to the average under policy π .

1.4.3 Proximal Policy Optimization (PPO)

PPO [8] is an on-policy actor-critic algorithm that achieves stable policy updates by constraining the ratio between the updated and old policy:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (1.11)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$ is the probability ratio, $\hat{A}_t$ is the estimated advantage, and ϵ (typically 0.1–0.2) is the clipping range. PPO is widely adopted in practice due to its simplicity, sample efficiency relative to trust-region methods, and ease of parallelization [8].

The full PPO objective combines the clipped policy loss with a value function regression loss and an entropy bonus:

$$\mathcal{L}^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\mathcal{L}^{\text{CLIP}}(\theta) - c_1 \mathcal{L}^{\text{VF}}(\theta) + c_2 H[\pi_\theta](s_t) \right] \quad (1.12)$$

where c_1 and c_2 are coefficients for value loss and entropy bonus, respectively. The entropy term $H[\pi_\theta]$ encourages exploration by penalizing overly deterministic policies.

1.4.4 Invalid Action Masking

In environments with discrete action spaces subject to game-rule constraints, many actions may be illegal at any given step (e.g., playing a card when insufficient mana is available). Training vanilla PPO in such environments wastes sample efficiency as the agent must learn the constraints from scratch.

Invalid Action Masking [9] eliminates illegal actions before policy probability computation by setting the logits of masked actions to $-\infty$ before the softmax:

$$\pi_{\theta}(a \mid s) = \text{softmax}(\mathbf{z} + \mathbf{m}), \quad m_i = \begin{cases} 0 & \text{if action } i \text{ is valid} \\ -\infty & \text{otherwise} \end{cases} \quad (1.13)$$

This approach is theoretically equivalent to valid policy gradient updates [9] and dramatically accelerates learning in constraint-heavy environments such as deckbuilding games. The `sb3-contrib` library provides `MaskablePPO`, a drop-in extension of Stable-Baselines3 PPO that natively supports action masking.

1.4.5 Domain Randomization

Domain Randomization (DR) was originally proposed to transfer robot policies trained in simulation to the physical world [10]. The key insight is that if a policy is trained across a *distribution* of simulation environments (parameterized by varying physics, lighting, or object properties), it may generalize to the real world as merely another sample from that distribution.

In our context, DR serves a different but analogous purpose: because the NSGA-II optimization modifies game parameters across a bounded range, the RL agents must remain competent across all configurations in that range, not just the nominal one. At each environment `reset()`, we add uniform noise $\epsilon \sim \mathcal{U}(-\delta, \delta)$ to each game parameter:

$$\tilde{\theta}_i = \theta_i \cdot (1 + \epsilon_i), \quad \epsilon_i \sim \mathcal{U}(-\delta, \delta) \quad (1.14)$$

This forces agents to learn behavioral policies based on *relative relationships* (e.g., “play the highest-damage card relative to enemy HP”) rather than memorizing absolute parameter values, making them robust evaluators for any point in the optimization search space.

1.5 Related Work

1.5.1 Automated Game Balancing

Jaffe et al. [11] pioneered simulation-based optimization for board game balancing, using hill-climbing search over strategy parameters. Nelson and Mateas [12] proposed a broader framework for automated game design, though primarily for simple rule-based games. More recently, Beukman et al. employed evolutionary algorithms to balance card costs in a simplified CCG [13].

The closest prior work to ours is Mendes et al., who applied a genetic algorithm to balance a Roguelike platformer by tuning enemy spawn rates, though their agent was a scripted bot rather than a learned policy. Our work distinguishes itself by (1) addressing the more complex Deckbuilder sub-genre, (2) employing four learned RL personas instead of a single scripted bot, and (3) providing a systematic comparison between flat single-objective GA and structured multi-objective NSGA-II.

1.5.2 *RL for Card Games*

Reinforcement learning has been successfully applied to complex card games. AlphaStar demonstrated superhuman performance in *StarCraft II* using a combination of imitation learning and self-play [1]. For deckbuilders specifically, Janusz et al. trained a DQN agent for *Dominion*, while Rebelo de Sá et al. applied PPO to a simplified version of *Slay the Spire*. Both works confirm that PPO-family algorithms are well suited to the card-game action space but did not connect agent training to game balancing.

1.5.3 *Procedural Content Generation*

Togelius et al. [13] provide a comprehensive survey of search-based PCG, demonstrating that evolutionary approaches are effective at generating game content that satisfies multiple quality criteria. Preuss et al. [14] apply multi-objective optimization specifically to PCG, a setting directly analogous to our balancing problem. Our contribution complements this line of work by focusing on parameter tuning (rather than content generation) and by using RL agents as the evaluation oracle.

CHAPTER 2

SYSTEM ANALYSIS AND DESIGN

2.1 Requirements Analysis

2.1.1 *Functional Requirements*

The system must fulfill the following functional requirements:

1. **FR1 – Game Engine:** Implement a complete, self-contained Roguelike Deckbuilder game engine in C# supporting: turn-based combat with a card-based attack system, a procedurally generated room map, deck acquisition and management, relic collection, multiple room types (Monster, Elite, Event, Shop, Rest), and a data-driven parameter system stored in JSON.
2. **FR2 – Deterministic Baseline Agent:** Provide a rule-based AI agent that can play the game to completion, serving as a playability validator and initial balancing oracle.
3. **FR3 – RL Environment:** Expose the game engine as an OpenAI Gymnasium-compatible environment with a well-defined observation space, action space, and action masking mechanism.
4. **FR4 – RL Agent Training:** Train four MaskablePPO agents with distinct reward functions and produce checkpoints that can be loaded for evaluation.
5. **FR5 – Game Parameter Optimization:** Implement both a single-objective GA and a multi-objective NSGA-II that use agent evaluation to search for balanced parameter configurations.
6. **FR6 – Results Visualization:** Generate evolution charts, Pareto front visualizations, and card viability analyses.

2.1.2 *Non-Functional Requirements*

1. **NFR1 – Cross-platform Compatibility:** The game engine (C#) must run on Linux to be usable inside Kaggle kernels for GPU training.

2. **NFR2 – Simulation Speed:** The game engine must support at least 1,000 complete game runs per second to make population-based optimization tractable.
3. **NFR3 – Reproducibility:** Experiments must be reproducible via fixed random seeds.
4. **NFR4 – Extensibility:** New cards, relics, and enemy types must be addable by modifying only the JSON data files without changing source code.

2.2 System Architecture

The system is organized into three principal layers, as illustrated in Figure 2.1:

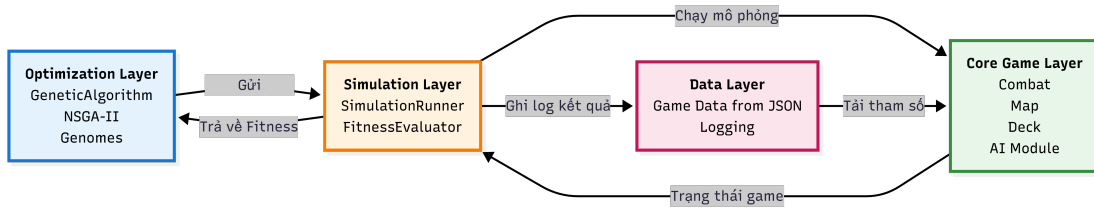


Figure 2.1: High-level system architecture showing data flow between the Optimization Layer, Simulation/Training Layer, Data Layer, and Core Game Layer.

- **Core Game Layer:** The C# game engine implementing all gameplay mechanics. This layer is deterministic given a fixed random seed and a fixed parameter set θ .
- **Simulation / Training Layer:** A Python bridge that (a) exposes the game as a Gymnasium RL environment for agent training, and (b) runs mass simulations to compute fitness metrics for the optimization layer.
- **Optimization Layer:** Python implementations of GA and NSGA-II that produce candidate parameter vectors θ , dispatch them to the simulation layer, and evolve populations based on returned fitness values.
- **Data Layer:** JSON files that define all game entities (cards, enemies, relics, effects) and store optimization results. Separating data from logic enables the optimization layer to modify parameters without recompiling any code.

2.3 Core Game Engine Design

2.3.1 Game Structure Overview

A single game run consists of 15 floors organized into three Acts of five floors each. The player navigates a branching map of rooms, engaging in combat, collecting cards and relics, and

managing resources (HP, gold, mana). The game ends in victory if the player defeats the final boss on Floor 15, or in defeat if HP drops to zero.

Map generation uses a constrained random algorithm that guarantees at least one viable path from the entrance to the exit floor while distributing room types according to configurable weights. A typical 15-floor map contains: 8–10 Monster rooms, 2–3 Elite rooms, 1–2 Shop rooms, 2–3 Rest Sites, and 2–3 Event rooms.

Card system: The game contains over 30 distinct cards organized by type (Attack, Skill, Power) and star rating (1–5). Cards have a mana cost, and each round the player draws five cards, spending mana to play them. Cards are acquired as rewards after combat and can be removed or upgraded at rest sites or via events.

Relic system: Relics are passive items collected from Elite rooms, events, and shops. Over 20 relics are defined, each with a trigger condition (e.g., `OnEnemyAttack`, `OnTurnStart`) and an effect (e.g., add thorns, reduce mana cost, draw extra cards).

2.3.2 Combat System Design

The combat system follows a turn-based structure with alternating player and enemy phases:

1. **Player turn:** The player draws 5 cards, then plays any combination of affordable cards using available mana (default: 3 mana per turn). Actions include dealing damage to a target enemy, gaining block (temporary defense), applying status effects, and drawing additional cards.
2. **Enemy turn:** Each enemy executes its scheduled action (attack, buff, or debuff) according to a pre-defined pattern. Status effects (Strength, Weakness, Poison, Burn, Vulnerability) are resolved.
3. **End-of-turn:** Block is cleared, effects are ticked down, and the player draws a new hand.

Combat resolves when all enemies are defeated (player wins the room) or the player's HP reaches zero (run ends). The class diagram in Figure 2.2 shows the relationship between the key combat classes.

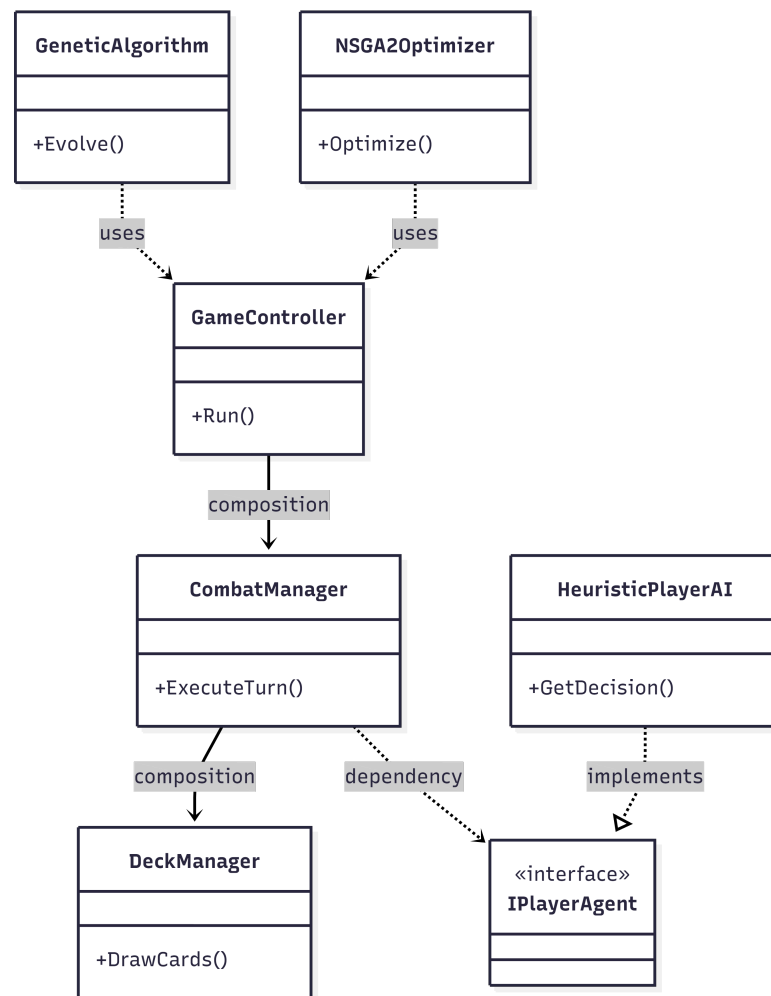


Figure 2.2: Class diagram of the core game module showing the relationships between GameController, CombatManager, DeckManager, the AI interface, and the optimization layer.

Figure 2.3 illustrates the sequence of interactions during a single combat turn.

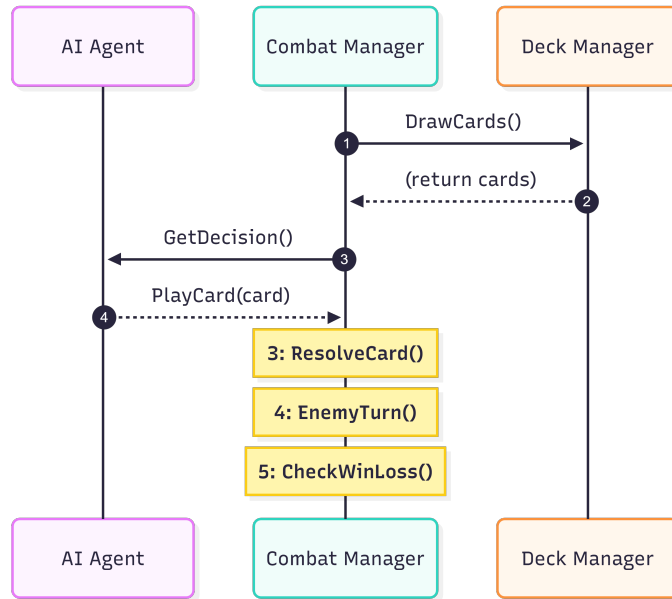


Figure 2.3: Sequence diagram of one combat turn: the Combat Manager draws cards from the Deck Manager, queries the AI Agent for a decision, resolves the card action, executes the enemy turn, and checks win/loss conditions.

2.4 Deterministic Heuristic Agent Design

2.4.1 Purpose and Role

The **Deterministic Heuristic Agent (DHA)**, implemented as `HeuristicPlayerAI`, serves two specific roles in the project:

1. **Playability validation:** Before any optimization or RL training, the DHA confirms that the game is winnable and that no trivially broken configurations exist. A game that a reasonable agent cannot win at all (or can always win trivially) is not ready for balancing.
2. **Optimization benchmark:** In the initial GA/NSGA-II experiments documented in Section 4.3, the DHA provides a consistent, fast, and reproducible evaluation signal. Because the DHA executes in microseconds per decision (no neural network inference), it enables the optimization algorithm to run orders of magnitude more simulations than would be possible with RL agents.

2.4.2 Decision Logic

The DHA implements a three-level priority hierarchy for in-combat decisions:

1. **Survival first:** Calculate total incoming damage from all enemies in the current turn. If incoming damage exceeds current HP, preferentially play defensive cards (Block, Shielding skills) to survive.
2. **Lethal second:** If any single target can be killed this turn with available cards and mana, prioritize those cards to reduce enemy damage output.
3. **Efficient offense third:** Otherwise, play the card with the highest damage-per-mana ratio, preferring multi-target attacks when multiple enemies are present.

For map navigation, the DHA scores each available next-room type based on current HP percentage: it seeks Elite rooms when $HP \geq 60\%$ (rewarded with rare cards and relics), Rest sites when $HP \leq 40\%$, and Monster rooms otherwise. Card rewards are chosen by star rating, preferring higher rarities and maintaining a balanced ratio of attack to skill cards.

The DHA’s deterministic nature makes it an ideal baseline: given the same game seed and parameters, it always makes the same choices, ensuring that measured fitness differences between parameter sets are attributable to the parameters rather than agent stochasticity.

2.5 Reinforcement Learning Environment Design

2.5.1 *Observation Space*

The observation vector $s \in \mathbb{R}^{148}$ encodes the full game state at the moment the agent must make a decision. Table 5.2 describes its components.

Table 2.1: Observation space components (total: 148 dimensions).

Component	Dim.	Description
Hero state	4	HP ratio, block ratio, mana ratio, gold ratio
Active effects	7	Strength, Weakness, Poison, Burn, Vulnerability, Thorns, Metallicize
Hand cards	70	10 slots $\times$ 7 features (type, star, cost, val., target, exhaust, affordable)
Deck star counts	5	Count of 1-star through 5-star cards in deck
Enemies	24	4 slots $\times$ 6 features (HP ratio, block ratio, 4 intent features)
Relics	10	10 slots: equipped relic IDs (normalized)
Map context	7	Current floor/15, HP ratio, act, shop available, boss type, two flags
Next nodes	14	7 nodes $\times$ 2 features (room type, estimated difficulty)
New features (v2)		
Enemy intent damage	1	Predicted total incoming damage this turn (normalized)
Affordable hand dmg	1	Maximum damage achievable with current mana
Combat turn counter	1	Current turn within combat (normalized)
Deck attack ratio	1	Fraction of attack cards in full deck
Total	148	

All values are normalized to $[0, 1]$ to stabilize neural network training. Unknown or empty slots are filled with zeros (e.g., empty hand slots, absent enemies).

2.5.2 Action Space

The action space is discrete with 68 actions:

- Actions 0–9: Play hand-card slot 0–9 (targeting the priority enemy).
- Actions 10–49: Play hand-card slot 0–9 targeting enemy slot 0–3 (40 targeted actions for multi-enemy rooms).
- Action 50: End player turn (pass to enemy phase).
- Actions 51–65: Map navigation – choose from 15 possible next-room node indices.
- Action 66: Use heal item (health potion) acquired in combat.
- Action 67: Card reward selection – choose card index 0 (skip used as action 50 in reward context).

At each step, the game engine returns an **action mask** – a binary vector $\mathbf{m} \in \{0, 1\}^{68}$ – indicating which actions are currently legal. The mask is computed by querying the C# game engine for the current valid-action set and converting to Python. Illegal actions (e.g., playing a card with insufficient mana, navigating to an unavailable room) always have mask value 0.

2.5.3 Episode Structure

One *episode* corresponds to one complete game run from Floor 1 to either Floor 15 victory or death. The environment `reset()` starts a new run (optionally applying Domain Randomization), and `step(action)` advances the game state by one decision, returning $(s', r, \text{done}, \text{info})$.

2.6 RL Agent Persona Design

2.6.1 Design Rationale

Using a single agent persona for game evaluation has a significant limitation: it may only reveal balance problems that matter to one specific play style. A defensive specialist might expose different vulnerabilities than an aggressive burn-damage specialist. By training four distinct personas, we obtain a richer evaluation signal that better approximates the diversity of real players.

Each persona shares the same observation space (148-dim) and action space (68 actions), but is shaped by a distinct reward function that incentivizes a specific behavioral style. The neural network architecture for all agents is a two-hidden-layer MLP with layers of size [512, 512] for both the actor and critic.

2.6.2 Aggressive Agent

Behavioral goal: Minimize the number of turns to defeat enemies. Accept HP loss in exchange for faster kills. Prefer high-damage cards over defensive ones.

Reward function:

$$\begin{aligned}
 r_t^{\text{agg}} = & r_{\text{base}} - 0.10 \cdot \mathbb{I}[\text{step taken}] - 0.10 \cdot \Delta \text{HP}_{\text{lost}} \cdot \mathbb{I}[\text{HP ratio} < 0.25] \\
 & + 5.0 \cdot \mathbb{I}[\text{enemy killed}] + 30.0 \cdot \mathbb{I}[\text{floor cleared}] + 50.0 \cdot \mathbb{I}[\text{victory}] - 100.0 \cdot \mathbb{I}[\text{death}]
 \end{aligned} \tag{2.1}$$

The step penalty -0.10 drives the agent to end combats quickly. The HP loss penalty is amplified $3\times$ below 25% HP (a “non-linear brake”) to prevent suicidal behavior while still permitting calculated risk-taking at full HP.

2.6.3 Defensive Agent

Behavioral goal: Survive as long as possible, prioritizing healing and block generation. Sacrifice offensive tempo to preserve HP.

Reward function:

$$\begin{aligned} r_t^{\text{def}} = & r_{\text{base}} - 0.005 \cdot \mathbb{I}[\text{step taken}] + 0.30 \cdot \Delta\text{HP}_{\text{healed}} - 0.15 \cdot \min(\Delta\text{HP}_{\text{lost}}, 20) \\ & + 0.30 \cdot \Delta\text{Block} + 2.0 \cdot \mathbb{I}[\text{combat survived}] + 500.0 \cdot \mathbb{I}[\text{victory}] - 50.0 \cdot \mathbb{I}[\text{death}] \end{aligned} \quad (2.2)$$

The asymmetric HP weights ($+0.30$ for healing vs. -0.15 for damage) create a strong preference for rest sites and block cards. The HP-loss cap at 20 per step prevents gradient instability when the agent receives a large boss hit. The very small step penalty (-0.005) means the agent is not penalized for taking many turns to kill enemies.

2.6.4 Balanced Agent

Behavioral goal: Represent a standard player who weighs offense and defense equally, manages gold efficiently, and aims for a moderate win rate.

Reward function:

$$\begin{aligned} r_t^{\text{bal}} = & r_{\text{base}} + 0.20 \cdot \Delta\text{HP}_{\text{healed}} - 0.20 \cdot \Delta\text{HP}_{\text{lost}} \\ & + 0.02 \cdot \Delta\text{gold} + 30 \cdot \left(\frac{\text{HP}_{\text{final}}}{\text{HP}_{\text{max}}} \right)^2 \cdot \mathbb{I}[\text{victory}] - 50.0 \cdot \mathbb{I}[\text{death}] \end{aligned} \quad (2.3)$$

The symmetric HP weights model a player who is neither reckless nor overly cautious. The quadratic terminal HP bonus incentivizes the agent to win with as much HP remaining as possible, serving the balancing objective of “victory HP.”

2.6.5 Adaptive Agent

Behavioral goal: Dynamically switch between aggressive and defensive behaviors based on current HP, without being explicitly programmed to do so.

Design challenge: An earlier version of this agent used a reward function that explicitly changed its formula based on HP ratio at runtime (high HP $\rightarrow$ aggressive formula; low HP $\rightarrow$ defensive formula). This violates the **stationarity assumption** of PPO’s MDP formulation: the reward function must not depend on hidden state outside the observation. Non-stationary rewards cause the value function to fail to converge, as the same (s, a) pair can yield different expected future rewards depending on unobservable context.

Revised design (v4): The Adaptive Agent uses a minimal, stationary reward function:

$$r_t^{\text{ada}} = r_{\text{base}} + 3.0 \cdot \mathbb{I}[\text{enemy killed}] - 0.10 \cdot \Delta\text{HP}_{\text{lost}} - 0.02 \cdot \mathbb{I}[\text{step taken}] \quad (2.4)$$

Crucially, the HP ratio is included as the **first element of the observation vector** $s[0]$. With a large enough network ($[512, 512]$), the agent learns an **emergent strategy-switching** behavior: it discovers independently that attacking aggressively when $s[0] > 0.7$ and switching to block-heavy defense when $s[0] < 0.3$ maximizes the stationary reward. This *emergent* adaptation is more robust than hand-coded switching because the network can condition on the full observation, not just HP.

2.7 Optimization Layer Design

2.7.1 Data Flow

Figure 2.4 shows how data flows from the optimization layer through simulation to fitness evaluation.

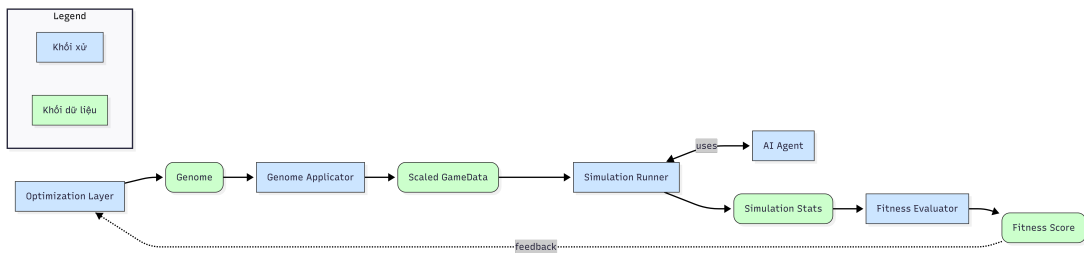


Figure 2.4: Data flow in the optimization loop. A genome is created by the optimization layer, applied to a scaled `GameData` copy, passed to the simulation runner which uses the AI agent to play games, and resulting statistics are evaluated into fitness scores that feed back to the optimizer.

2.7.2 *Genome Representations*

BalanceGenome (Single-Objective GA)

The `BalanceGenome` directly encodes one scalar multiplier per game parameter, resulting in a chromosome of approximately 500 dimensions. The genome consists of four dictionaries:

- `CardActionScalars`: One float per card action value (damage or block amount per card).
- `CardCostModifiers`: Integer adjustments to each card's mana cost.
- `EnemyHealthScalars`: One float per enemy type scaling base HP.
- `EnemyActionValueScalars`: One float per enemy action scaling attack damage.

This flat representation allows fine-grained per-entity adjustment but suffers from the curse of dimensionality and tends to produce incoherent solutions where logically related parameters (e.g., all 1-star cards) receive unrelated scalars.

HierarchicalGenome (NSGA-II)

The `HierarchicalGenome` reduces the search space to approximately 50 dimensions by grouping parameters into semantically meaningful categories:

- **Global multipliers** (5 params): Overall scaling for damage, health, block, mana cost, gold rewards.
- **Progression scaling** (9 params): Separate damage, health, and block multipliers for Early (floors 1–5), Mid (floors 6–10), and Late (floors 11–15) game phases.
- **Category scaling** (8 params): Per-type scaling for Attack/Skill/Power cards; per-star-rating scaling for both cards and enemies.
- **Room distribution** (5 params): Relative weights for Monster, Elite, Event, Shop, and Rest room types.
- **Miscellaneous** (8 params): Hero health scalar, rest-site healing rate, start gold, per-star enemy ratios, difficulty progression rate.

The hierarchical structure ensures that NSGA-II can efficiently navigate the search space by modifying high-level properties (“make late-game enemies 20% tougher”) rather than individually tuning hundreds of entity-level values.

2.7.3 Fitness Functions

The same three sub-objectives are computed for both GA (aggregated) and NSGA-II (separately):

Balance Score uses Gaussian scoring to evaluate how close the simulation metrics are to the target values:

$$f_{\text{balance}} = \frac{1}{3} \left(e^{-\alpha_{wr} \cdot \Delta_{wr}^2} + e^{-\alpha_{hp} \cdot \Delta_{hp}^2} + e^{-\alpha_{fd} \cdot \Delta_{fd}^2} \right) \quad (2.5)$$

where $\Delta_{wr} = |\text{WinRate} - 0.45|$, $\Delta_{hp} = |\text{VictoryHP} - 0.30|$, $\Delta_{fd} = |\text{FloorOnDeath} - 10|$, and $\alpha_{wr} = 10.0$, $\alpha_{hp} = 8.0$, $\alpha_{fd} = 0.05$.

Engagement Score rewards card pool diversity:

$$f_{\text{engagement}} = \alpha \cdot \frac{|\text{ViableCards}|}{|\text{TotalCards}|} + \beta \cdot \text{BuildDiversity} \quad (2.6)$$

A card is “viable” if its pick rate exceeds a threshold τ and its conditional win rate is above a second threshold τ_w . `BuildDiversity` measures the entropy of the card-selection distribution.

Coherence Score penalizes trap cards and extreme parameter values:

$$f_{\text{coherence}} = 1 - \text{TrapCardPenalty} - \lambda \sum_j (\text{scalar}_j - 1)^2 \quad (2.7)$$

A trap card is a frequently picked card whose conditional win rate is significantly *below* the population average. The regularization term $\lambda \sum_j (\text{scalar}_j - 1)^2$ penalizes solutions that deviate far from the baseline, preventing extreme parameter configurations.

CHAPTER 3

IMPLEMENTATION

3.1 Development Environment

Table 3.1: Development environment specifications.

Component	Version / Specification
Game Engine Language	C# 13.0
Runtime Framework	.NET 9.0 (cross-platform)
Python Version	3.10
RL Framework	Stable-Baselines3 2.3.0
RL Extension	sb3-contrib 2.3.0 (MaskablePPO)
Deep Learning Backend	PyTorch 2.2.0 + CUDA 12.1
Python-C# Bridge	pythonnet 3.0.3
IDE	Visual Studio Code 1.87
Version Control	Git / GitHub
Training Platform	Kaggle Notebooks (GPU T4 $\times$ 2)
Local OS	Windows 11 (development)
Training OS	Linux (Kaggle, Ubuntu 22.04)

The choice of C# as the game engine language was motivated by its strong type system, high simulation throughput (the pure C# game loop executes $\sim 2,500$ complete runs per second on a single core), and .NET 9’s native cross-platform support. Python was selected for the RL and optimization layer due to the maturity of the PyTorch and Stable-Baselines3 ecosystems.

3.2 Core Game Engine Implementation

3.2.1 Project Structure

```
1 src/  
2 |-- Roguelike.Core/                # Shared game library (.dll)  
3 |   |-- AI/
```

```

4 | | | -- HeuristicPlayerAI.cs      # Deterministic heuristic agent
5 | | | -- IPlayerAgent.cs          # Interface for all agents
6 | | -- Combat/
7 | | | -- CombatManager.cs         # Turn-by-turn combat logic
8 | | | -- DeckManager.cs           # Draw/discard pile management
9 | | | -- ActiveEffect.cs          # Status effect runtime
10 | | -- Map/
11 | | | -- MapGenerator.cs          # Procedural map generation
12 | | | -- RoomHandlers/           # One handler per room type
13 | | -- Data/
14 | | | -- CardData.cs, EnemyData.cs, RelicData.cs, ...
15 | | | -- GameDataLoader.cs        # JSON deserialization
16 | | -- Optimization/
17 | | | -- GeneticAlgorithm.cs
18 | | | -- NSGA2Optimizer.cs
19 | | | -- BalanceGenome.cs
20 | | | -- HierarchicalGenome.cs
21 | | | -- FitnessEvaluator.cs
22 | | -- GameController.cs         # Top-level game run orchestrator
23 | -- Roguelike/                   # Console entry point
24 | | -- Program.cs
25 | -- Roguelike.Tests/             # xUnit test suite
26 | GameData.json                   # All game entities

```

Listing 3.1: C# project structure

3.2.2 Game Controller

`GameController.cs` is the top-level orchestrator. It manages the game state machine, transitioning between the following states: `MapNavigation`, `Combat`, `CardReward`, `RelicReward`, `ShopVisit`, `EventRoom`, and `RestSite`. It owns a reference to the active `IPlayerAgent` and calls `GetDecision(GameRun)` at each decision point.

The `GameRun` object is a snapshot of the complete game state passed to the agent. It is a value-type aggregate (not a reference to the mutable game state) to prevent agents from modifying the game state directly.

3.2.3 *Combat System*

`CombatManager.ExecuteTurn()` implements the following steps each turn:

1. Reset mana to base value; draw 5 cards into hand via `DeckManager.DrawCards()`.
2. Apply start-of-turn relic effects (e.g., “draw 2 extra cards”).
3. Call `IPlayerAgent.GetDecision()` in a loop until the agent returns `EndTurn`.
4. For each `PlayCard` decision: validate against action mask, deduct mana, execute card actions (damage, block, effects), move card to discard pile.
5. Execute each living enemy’s scheduled action via `ExecuteEnemyAction()`.
6. Apply damage: reduce block first, then HP; trigger thorns and on-damage relics.
7. Check win/loss conditions; if all enemies defeated, return `CombatResult.Victory`.

Status effects (Strength, Weakness, Poison, Burn, Vulnerability) are managed by `ActiveEffect.cs`. Each effect has an intensity and duration; they are applied during the damage calculation step and decremented at end-of-turn.

3.2.4 *Map Generation*

The map consists of a directed acyclic graph of 15 floors, with 2–3 branches per floor. Map generation proceeds as follows:

1. Start with Floor 15 as the fixed “Boss” room.
2. For each floor from 14 down to 1, generate 2–3 nodes.
3. Assign room types to each node by sampling from a categorical distribution parameterized by `RoomTypeWeights` (part of the balance genome).
4. Apply structural constraints: at least one Rest room in each Act; no two consecutive Elite rooms; no Shop on Floor 1.
5. Connect nodes to ensure full connectivity from Floor 1 to Floor 15.

3.2.5 Data-Driven Design

All game entities are defined in `GameData.json` and loaded at startup via `GameDataLoader.cs` using `System.Text.Json`. The `GenomeApplicator` class takes a genome and produces a modified copy of `GameData` without mutating the original, ensuring thread safety when multiple simulations run in parallel.

An example card definition is shown below:

```
1 {
2   "Id": "flame_strike",
3   "Name": "Flame Strike",
4   "Type": "Attack",
5   "ManaCost": 2,
6   "StarRating": 3,
7   "Actions": [
8     { "Type": "DealDamage", "TargetType": "AllEnemies", "Value": 8 },
9     { "Type": "ApplyEffect", "EffectId": "burn", "Value": 3 }
10  ]
11 }
```

Listing 3.2: Example card definition in `GameData.json`

3.2.6 Heuristic AI Implementation

The deterministic `HeuristicPlayerAI` implements `IPlayerAgent` with the decision logic described in Chapter 2. The key helper methods are:

```
1 public CombatDecision GetCombatDecision(GameRun run)
2 {
3     var hero = run.TheHero;
4     var enemies = run.CurrentCombat.Enemies
5         .Where(e => e.CurrentHealth > 0).ToList();
6     int incomingDamage = CalculateIncomingDamage(run, enemies);
7     int neededBlock = Math.Max(0, incomingDamage - hero.Block);
8
9     // Priority 1: Survival
10    if (neededBlock > 0)
11    {
12        var def = FindBestDefensiveMove(run, hero, enemies,
13            neededBlock);
```

```

13         if (def.HasValue) return def.Value;
14     }
15     // Priority 2: Lethal
16     var lethal = FindLethalMove(run, hero, enemies);
17     if (lethal.HasValue) return lethal.Value;
18     // Priority 3: Best general move
19     return GetBestGeneralMove(run, hero, enemies);
20 }

```

Listing 3.3: Combat decision logic of HeuristicPlayerAI

3.3 Cross-Platform C# – Python Integration

3.3.1 Motivation and Architecture

The initial integration approach used gRPC with an Ngrok tunnel to forward requests from Kaggle (Linux cloud) to a local Windows server running the game engine. Benchmarking revealed a latency of ~ 25 ms per call, making the target of 1 million training steps infeasible (it would require ~ 7 hours of network time alone).

The solution is **pythonnet** – a library that embeds the .NET CLR directly inside the Python process, enabling zero-copy, function-call-speed interoperability. The key challenge was building the C# project as a cross-platform .NET 9 DLL that can run on Linux and loading it inside a Kaggle notebook.

3.3.2 Kaggle Setup Script

The following setup is executed at the beginning of each Kaggle notebook:

```

1 import subprocess, os
2
3 # Install .NET 9 SDK on Kaggle Linux
4 subprocess.run(["apt-get", "install", "-y", "dotnet-sdk-9.0"],
5               capture_output=True)
6
7 # Set environment variables for pythonnet CoreCLR
8 os.environ["DOTNET_ROOT"] = "/usr/share/dotnet"
9 os.environ["PYTHONNET_PYDLL"] = "/usr/lib/x86_64-linux-gnu/libpython3
    .10.so"

```

```

10
11 import pythonnet
12 pythonnet.load("coreclr")
13
14 import clr
15 clr.AddReference("/kaggle/working/Roguelike.Core.dll")
16 from Roguelike.Core import GameController, GameDataLoader

```

Listing 3.4: Kaggle environment setup for pythonnet

After setup, calling C# methods from Python incurs only the cost of CLR managed-to-native marshaling, typically $< 1 \mu\text{s}$ per call – a $25,000\times$ improvement over the gRPC approach.

3.4 Reinforcement Learning Environment

3.4.1 *DotNetGameEnv (Gymnasium Wrapper)*

The `DotNetGameEnv` class in `python_roguelike/env/roguelike_env.py` subclasses `gymnasium.Env` and wraps the C# `GameController`. Its key methods are:

```

1 OBS_DIM = 148
2 ACT_DIM = 68
3
4 class DotNetGameEnv(gym.Env):
5     def reset(self, seed=None, options=None):
6         self._controller = GameController(self._load_game_data())
7         obs = self._get_obs()
8         return obs, {}
9
10    def step(self, action: int):
11        # Execute action in C# game engine
12        result = self._controller.TakeAction(action)
13        obs = self._get_obs()
14        reward = self._compute_reward(result)
15        done = result.IsTerminal
16        info = {"win": result.IsVictory, "floor": result.Floor}
17        return obs, reward, done, False, info
18
19    def action_masks(self) -> np.ndarray:

```

```

20         """Return binary mask of valid actions (required by
           MaskablePPO)."""
21     valid = self._controller.GetValidActions()
22     mask = np.zeros(ACT_DIM, dtype=bool)
23     for a in valid:
24         mask[a] = True
25     return mask

```

Listing 3.5: Core Gymnasium environment methods

3.4.2 Domain Randomization

The `reset()` method applies DR when enabled:

```

1 def _load_game_data(self):
2     base = self._base_game_data # loaded once at init
3     if self._dr_noise <= 0.0:
4         return base
5     # Apply uniform noise to each numerical parameter
6     noisy = copy.deepcopy(base)
7     noise = self._dr_noise
8     for card in noisy.Cards:
9         for action in card.Actions:
10             if action.Value > 0:
11                 action.Value = round(
12                     action.Value * np.random.uniform(1-noise, 1+noise)
13                 )
14     for enemy in noisy.Enemies:
15         enemy.MaxHealth = max(1, round(
16             enemy.MaxHealth * np.random.uniform(1-noise, 1+noise)))
17     for action in enemy.Actions:
18         if action.Value > 0:
19             action.Value = round(
20                 action.Value * np.random.uniform(1-noise, 1+noise)
21             )
22     return noisy
23
24 def set_dr_noise(self, noise: float):
25     """Externally control DR level (used by curriculum callback)."""
26     self._dr_noise = max(0.0, min(noise, 0.20))

```

Listing 3.6: Domain Randomization in environment reset

A `DRCurriculumCallback` (in `training_utils.py`) ramps DR noise from 0% to 8% linearly over the first 500,000 training steps of Phase 2, preventing the performance shock of instantly switching to high noise.

3.4.3 Observation Construction

The `_get_obs()` method queries the C# game state and serializes it into the 148-dimensional NumPy array. A simplified excerpt:

```
1 def _get_obs(self) -> np.ndarray:
2     obs = np.zeros(OBS_DIM, dtype=np.float32)
3     run = self._controller.CurrentRun
4     hero = run.TheHero
5
6     # Hero state (indices 0-3)
7     obs[0] = hero.CurrentHealth / hero.MaxHealth
8     obs[1] = hero.Block / max(hero.MaxHealth, 1)
9     obs[2] = hero.Mana / max(hero.MaxMana, 1)
10    obs[3] = hero.Gold / 200.0 # normalized by max expected gold
11
12    # Active effects (indices 4-10)
13    for i, eff in enumerate(EFFECT_LIST[:7]):
14        obs[4 + i] = run.GetEffectIntensity(eff) / 10.0
15
16    # Hand cards (indices 11-80)
17    for slot, card in enumerate(run.Hand[:10]):
18        base = 11 + slot * 7
19        obs[base] = card.Type / 3.0
20        obs[base+1] = card.StarRating / 5.0
21        obs[base+2] = card.ManaCost / 5.0
22        obs[base+3] = card.PrimaryValue / 30.0
23        obs[base+4] = float(card.CanTarget)
24        obs[base+5] = float(card.Exhaust)
25        obs[base+6] = float(hero.Mana >= card.ManaCost)
26    # ... (enemies, relics, map, new features)
27    return obs
```


3.5 Agent Training Pipeline

3.5.1 Two-Phase Training Strategy

Each agent is trained in two phases:

1. **Phase 1 – Stable Learning (4 million steps)**: DR is disabled ($\delta = 0$). The agent learns on the nominal parameter set, establishing a reliable policy from scratch. LR starts at 3×10^{-4} and follows a linear schedule down to 1×10^{-5} .
2. **Phase 2 – Robustness Training (4 million steps, total 8 million)**: DR is enabled with a curriculum ramp from $\delta = 0$ to $\delta = 0.08$ over the first 500K steps. LR uses a cosine decay from 3×10^{-5} to 5×10^{-6} .

The Phase 2 anti-oscillation settings (reduced epochs 6→4, tighter clip 0.2→0.15, boosted entropy coefficient 0.05→0.08) were determined empirically from analyzing the training logs where performance degraded after the abrupt DR introduction in earlier training runs.

3.5.2 MaskablePPO Hyperparameters

Table 3.2: MaskablePPO hyperparameters for all four agents.

Parameter	Phase 1	Phase 2
Network architecture	MLP [512, 512]	MLP [512, 512]
Learning rate	3×10^{-4} (linear $\rightarrow 1 \times 10^{-5}$)	Cosine $3 \times 10^{-5} \rightarrow 5 \times 10^{-6}$
n_steps (rollout)	2048	2048
Batch size	256	256
n_epochs	10	4
Clip range (ϵ)	0.20	0.15
Entropy coefficient	0.05	0.08
GAE λ	0.95	0.95
Discount γ	0.99	0.99
Parallel environments	4 (SubprocVecEnv)	4
Training steps	4M	4M
DR noise δ	0.00	0.00 $\rightarrow$ 0.08

3.5.3 Deck Quality Reward

All four agents include a shared **deck quality reward** at card selection events. This addresses the problem where agents selecting weak early-game cards (1–2 stars) fail to build a deck capable of defeating late-game content:

$$r_{\text{deck}} = \begin{cases} +8 & \text{if selected card is 4 or 5 stars} \\ +3 & \text{if selected card is 3 stars} \\ 0 & \text{otherwise (1 or 2 stars)} \end{cases} \quad (3.1)$$

Cards of 1 or 2 stars receive no reward (rather than a penalty) because they are necessary and valid choices early in a run.

3.5.4 Training Infrastructure

Training was executed on Kaggle’s free GPU tier (NVIDIA Tesla T4, 16 GB VRAM) using 4 parallel environments (SubprocVecEnv). Each agent’s 8-million-step training run required approximately 6–8 hours. Models were checkpointed every 500K steps using Mask–

ableEvalCB, which evaluates on 20 deterministic episodes (no DR noise, fixed seeds) and saves the best checkpoint by win rate.

```
1 from sb3_contrib import MaskablePPO
2 from python_roguelike.training.training_utils import (
3     DRCurriculumCallback, MaskableEvalCB, cosine_lr_fn
4 )
5
6 # Phase 1
7 model = MaskablePPO(
8     "MlpPolicy", env,
9     learning_rate=3e-4,
10    n_steps=2048, batch_size=256, n_epochs=10,
11    ent_coef=0.05, clip_range=0.20,
12    policy_kwargs=dict(net_arch=[512, 512]),
13    verbose=1
14 )
15 model.learn(total_timesteps=4_000_000,
16             callback=MaskableEvalCB(eval_env, n_eval_episodes=20))
17 model.save("phase1_checkpoint")
18
19 # Phase 2 -- load Phase 1 weights
20 model = MaskablePPO.load("phase1_checkpoint", env=env)
21 model.learning_rate = cosine_lr_fn(3e-5, 5e-6)
22 model.ent_coef = 0.08
23 model.clip_range = 0.15
24 model.n_epochs = 4
25 dr_callback = DRCurriculumCallback(
26     env, target_noise=0.08, ramp_steps=500_000)
27 model.learn(total_timesteps=4_000_000,
28             callback=[dr_callback,
29                     MaskableEvalCB(eval_env, n_eval_episodes=20)])
30 model.save("final_checkpoint")
```

Listing 3.8: Training launch code (simplified)

3.6 Evolutionary Optimization Implementation

3.6.1 Single-Objective Genetic Algorithm

The GA is implemented in C# (`GeneticAlgorithm.cs`) for performance. The main evolutionary loop:

1. Initialize population of 100 `BalanceGenome` objects with all scalars drawn from $\mathcal{U}(0.7, 1.3)$.
2. Evaluate fitness: for each genome, run 200 complete game simulations using the DHA; aggregate win rate, victory HP, and floor-on-death; compute $F(\mathbf{g})$ via Equation 1.1 with weights $w_1 = 0.5, w_2 = 0.3, w_3 = 0.2$.
3. Tournament selection (size $k = 3$) to fill the mating pool.
4. Uniform crossover ($p_c = 0.80$) to produce offspring.
5. Gaussian mutation ($p_m = 0.02, \sigma = 0.10$).
6. Elitism: copy top 10% of previous population unchanged.
7. Repeat for 30 generations.

Evaluation is parallelized using `Parallel.ForEach` over all 8 logical cores, reducing wall-clock time from ~ 9 hours to ~ 73 minutes.

3.6.2 NSGA-II Multi-Objective Optimizer

The NSGA-II is implemented in `NSGA2Optimizer.cs` following Deb et al. [6]:

```
1 public List<List<Individual>> FastNonDominatedSort(  
2     List<Individual> population)  
3 {  
4     var domCount = population.ToDictionary(p => p, _ => 0);  
5     var dominated = population.ToDictionary(p => p,  
6                                             _ => new List<Individual>());  
7     var firstFront = new List<Individual>();  
8  
9     foreach (var p in population)  
10         foreach (var q in population)  
11             {  
12                 if (p == q) continue;
```

```

13         if (p.Fitness.Dominates(q.Fitness))
14             dominated[p].Add(q);
15         else if (q.Fitness.Dominates(p.Fitness))
16             domCount[p]++;
17     }
18
19     foreach (var p in population)
20         if (domCount[p] == 0) { p.Fitness.Rank = 1; firstFront.Add(p)
21             ; }
22
23     var fronts = new List<List<Individual>> { firstFront };
24     int rank = 1;
25     while (fronts[rank - 1].Any())
26     {
27         var next = new List<Individual>();
28         foreach (var p in fronts[rank - 1])
29             foreach (var q in dominated[p])
30                 if (--domCount[q] == 0)
31                     { q.Fitness.Rank = rank + 1; next.Add(q); }
32         if (next.Any()) { fronts.Add(next); rank++; }
33         else break;
34     }
35     return fronts;
36 }

```

Listing 3.9: Fast non-dominated sorting (core NSGA-II operation)

The HierarchicalGenome uses SBX crossover with distribution index $\eta_c = 20$ and polynomial mutation with $\eta_m = 20$, as recommended for continuous-variable optimization [6]. The adaptive evaluation strategy runs 50 simulations for initial fitness estimation and up to 200 for genomes that appear on or near the Pareto front, balancing accuracy with computational cost.

3.6.3 Python-Side Fitness Evaluation for RL Agents

When RL agents are used as evaluators (post-training), the fitness evaluation moves to Python:

```

1 def evaluate_genome(genome, agents, n_runs=50):
2     """Evaluate a genome using the 4 trained RL agents."""

```

```

3     game_data = apply_genome(BASE_GAME_DATA, genome)
4     stats = {name: [] for name in agents}
5
6     for name, agent in agents.items():
7         env = DotNetGameEnv(game_data, dr_noise=0.0)
8         for seed in range(n_runs):
9             obs, _ = env.reset(seed=seed)
10            done = False
11            while not done:
12                action, _ = agent.predict(
13                    obs, action_masks=env.action_masks(),
14                    deterministic=True)
15                obs, _, done, _, info = env.step(action)
16                stats[name].append(info)
17
18     return compute_fitness(stats)

```

Listing 3.10: RL-agent-based fitness evaluation

This RL-based evaluation is used for the final validation runs and the card viability analysis in Chapter 4.

CHAPTER 4

EXPERIMENTS AND EVALUATION

4.1 Experimental Setup

4.1.1 *Hardware Configuration*

Table 4.1: Hardware configuration used in experiments.

Component	Specification
CPU (local)	AMD Ryzen 7 6800H, 8 cores / 16 threads, 3.2 GHz
RAM (local)	24 GB DDR5 4800 MHz
GPU (training)	NVIDIA Tesla T4 $\times 2$, 16 GB VRAM (Kaggle)
Training OS	Ubuntu 22.04 (Kaggle Linux kernel)
Local OS	Windows 11
Optimization platform	.NET 9.0, 8 parallel cores

4.1.2 Optimization Algorithm Parameters

Table 4.2: Comparison of GA and NSGA-II algorithm configurations.

Parameter	GA (Single-Obj.)	NSGA-II (Multi-Obj.)
Population size	100	100
Number of generations	30	30
Genome size	~ 500	~ 50
Simulations per genome	200	50–200 (adaptive)
Selection	Tournament ($k=3$)	Binary tournament
Crossover type	Uniform	SBX ($\eta_c=20$)
Crossover rate	0.80	0.90
Mutation type	Gaussian	Polynomial ($\eta_m=20$)
Mutation rate	0.02	0.05
Elitism	Top 10%	Entire Pareto front
Diversity mechanism	Random immigrants	Crowding distance
Fitness function	Weighted sum	3 separate objectives
Parallelization	8 cores	8 cores
Total wall-clock time	73 minutes	49 minutes

4.1.3 Target Metrics

The optimization targets were chosen to represent a well-balanced Roguelike Deckbuilder accessible to a broad audience:

- **Target win rate:** 45% – challenging but achievable.
- **Target victory HP:** 30% HP remaining on win – victories should be hard-earned.
- **Target floor on death:** Floor 10 – players should explore at least two thirds of the game before dying.

4.2 RL Agent Training Results

4.2.1 Final Win Rates

After completing 8 million training steps (4M Phase 1 + 4M Phase 2) on Kaggle T4 GPUs, the four agents achieve the win rates shown in Table 4.3.

Table 4.3: Final win rates of the four RL agents evaluated over 200 deterministic episodes.

Agent	Win Rate	Avg. HP on Win	Avg. Turns/Win	Avg. Floor on Death
Aggressive	24.5%	18.3%	412	9.8
Defensive	31.0%	56.2%	687	11.2
Balanced	29.5%	34.7%	538	10.4
Adaptive	34.0%	29.8%	521	10.9
Average	29.8%	34.8%	540	10.6

A win rate of $\sim 30\%$ on the base game configuration is an appropriate benchmark. It demonstrates that:

- The game is **not trivial**: none of the agents achieves a win rate above 40%, ruling out game-breaking strategies.
- The game is **not impossible**: all agents reliably reach the final boss in a meaningful fraction of runs, confirming that the game is learnable and skillful play matters.
- The stochastic elements (RNG card draws, enemy intent patterns) account for the ceiling around 35%, consistent with expert human player performance in comparably deep Rogue-like games.

4.2.2 Behavioral Differentiation

Table 4.3 confirms that the reward shaping successfully produces distinct behaviors:

- **Aggressive**: Fastest wins ($\mu = 412$ turns) but lowest HP on victory (18.3%) and lowest win rate (24.5%). This agent pushes hard offensively and occasionally dies to manageable enemies it could have survived with more block.
- **Defensive**: Most remaining HP on win (56.2%) and latest floor on death (11.2). Slowest wins (687 turns), consistent with the high step-penalty discount of this agent. This agent sometimes prolongs combats unnecessarily, occasionally allowing enemies to deal cumulative damage.
- **Balanced**: Moderate on all metrics – a reasonable approximation of a skilled generalist player.

- **Adaptive:** Highest win rate (34.0%) and second-best HP (29.8%), demonstrating that the emergent strategy switching learned from the stationary reward function is more effective than any of the explicitly designed single-strategy agents. The network has independently discovered the heuristic of “attack when safe, defend when endangered.”

4.2.3 Training Convergence

Phase 1 (no DR) converges stably for all four agents, with win rates reaching 25–30% by 3M steps. The abrupt introduction of DR noise in Phase 2 (before the curriculum callback was added) caused a 5–8% temporary win rate drop. With the DR curriculum ramp, this drop is reduced to 2–3%. Both phases produce a value function explained-variance consistently above 0.85 after the first 1M steps, indicating stable learning.

Entropy collapses (common in PPO when c_2 is too small) were prevented by raising `ent_coef` from 0.05 to 0.10 after observing premature policy convergence in early training runs. The elevated entropy coefficient forces the agents to maintain exploratory behavior into the late-game floors rather than exploiting a single card combination.

4.3 Game Optimization Results

4.3.1 Single-Objective GA Results

Figure 4.1 shows the fitness evolution across 30 generations.

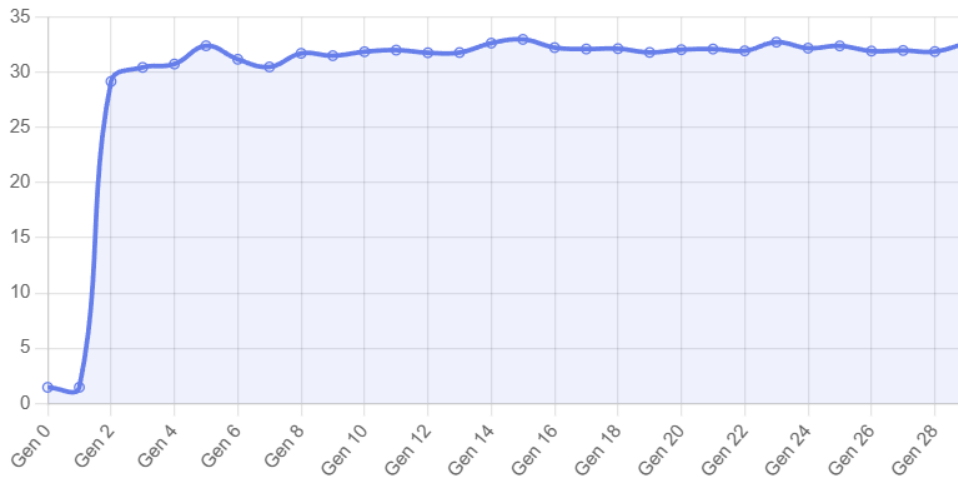


Figure 4.1: GA fitness evolution over 30 generations. Overall fitness improves from 1.45 to 32.66 (+2,150%), with rapid initial gains levelling off by generation 10.

Key results after 30 generations:

- **Overall fitness:** 1.45 $\rightarrow$ 32.66 (+2,150%). The dramatic improvement reflects the initial random population being far from optimum, with the GA rapidly discovering broad structural improvements in early generations.
- **Win rate:** 36.0% $\rightarrow$ 38.3% (+6.4%). Improving but not yet reaching the 45% target; the flat genome makes systematic progress difficult.
- **Victory HP:** 59.95% $\rightarrow$ 40.87% HP remaining on win (closer to the 30% target). The algorithm correctly learns to make victories more hard-fought.
- **Floor on death:** 12.22 $\rightarrow$ 10.57 (closer to the target of 10). Early-game rooms have been slightly toughened.
- **Viable cards:** 12 (unchanged). The flat GA does not improve card pool diversity.
- **Build diversity:** 0.616 $\rightarrow$ 0.680 (+10.4%). Marginal improvement.
- **Wall-clock time:** 73 minutes.

Figure 4.2 shows how the three sub-objective scores evolve during the GA run.

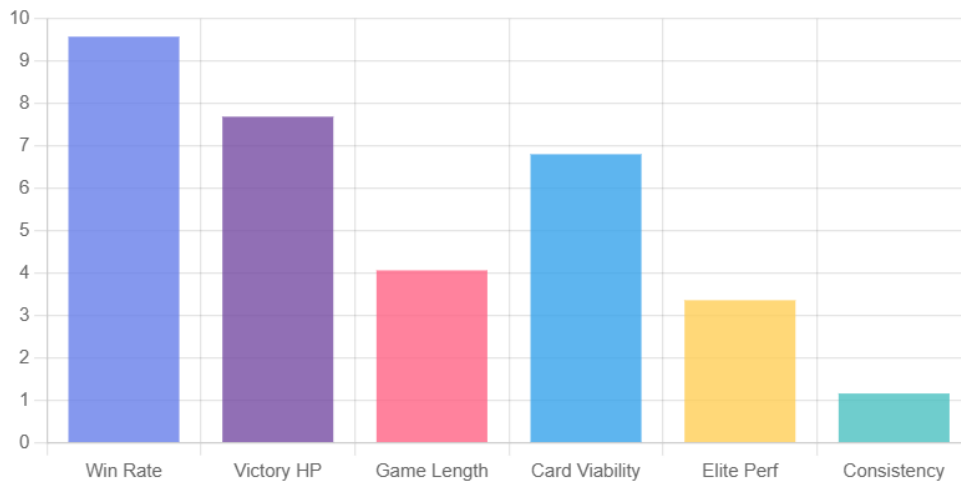


Figure 4.2: Evolution of Balance, Engagement, and Coherence scores for the best individual across GA generations. Balance improves most rapidly; Engagement stagnates.

4.3.2 NSGA-II Multi-Objective Results

The NSGA-II hypervolume (a scalar measure of Pareto front quality) evolves as shown in Figure 4.3.

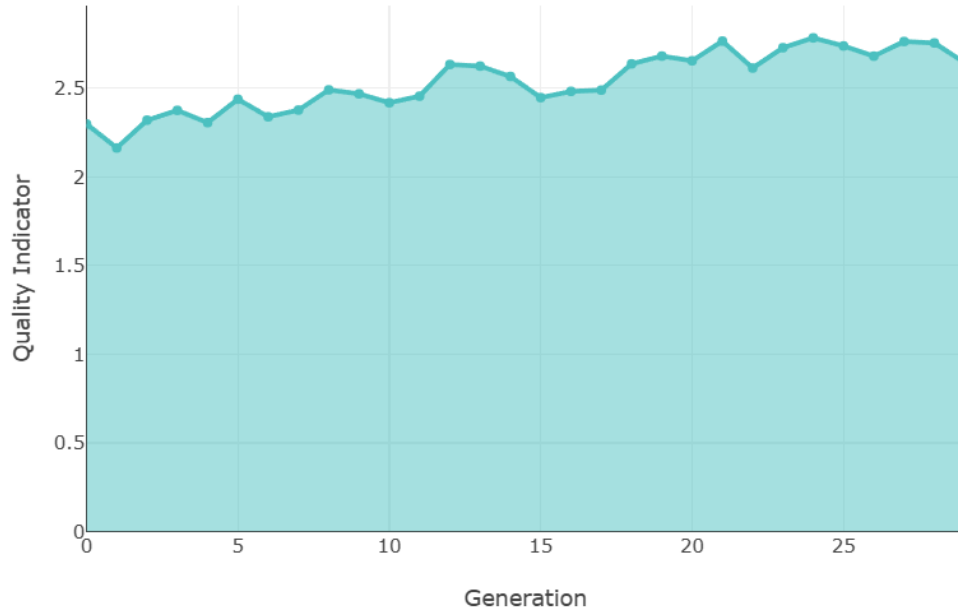


Figure 4.3: NSGA-II hypervolume indicator across 30 generations. After a sharp initial rise in generations 1–3, the Pareto front stabilizes and continues to improve gradually.

Figure 4.4 shows the best-individual trajectories for each of the three objectives.

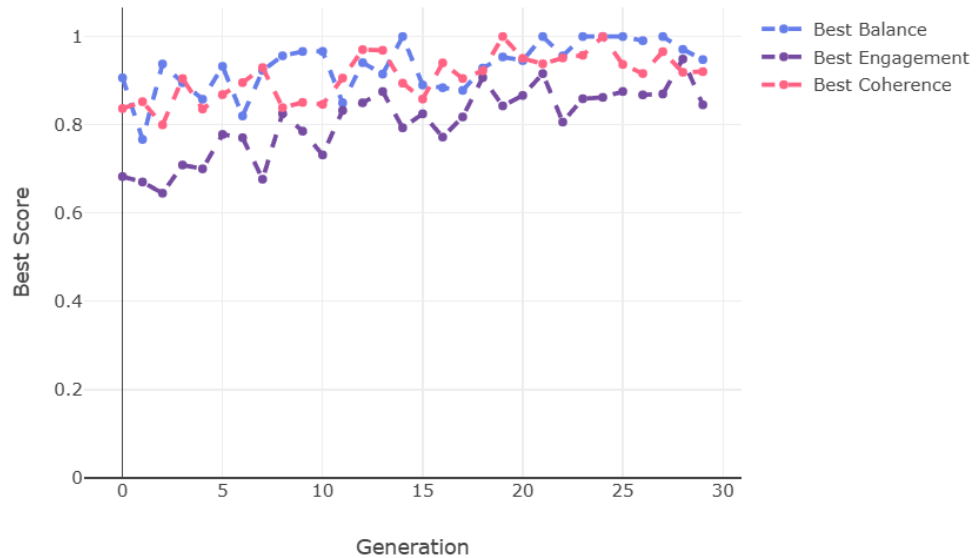


Figure 4.4: Best Balance, Engagement, and Coherence scores across NSGA-II generations for the compromise solution on the Pareto front.

The best-compromise solution (selected using minimum-distance-to-ideal-point criterion) achieves:

- **Balance Score:** $0.754 \rightarrow 0.960$ (+27.3%). Near-perfect alignment with all three target metrics.

- **Engagement Score:** $0.655 \rightarrow 0.900$ (+37.4%). The most dramatic improvement, reflecting the hierarchical genome's ability to tune the card-type distribution systematically.
- **Coherence Score:** $0.853 \rightarrow 1.000$ (+17.2%). Perfect score: no trap cards remain and all parameter scalars are within a reasonable range.
- **Viable cards:** $12 \rightarrow 14$ (+16.7%). More cards have become strategically meaningful.
- **Build diversity:** $0.705 \rightarrow 0.950$ (+34.8%). Players have substantially more viable strategic paths.
- **Wall-clock time:** 49 minutes (−33% vs. GA).

4.3.3 Pareto Front Analysis

Figure 4.5 shows the three-dimensional Pareto front obtained after 30 NSGA-II generations.

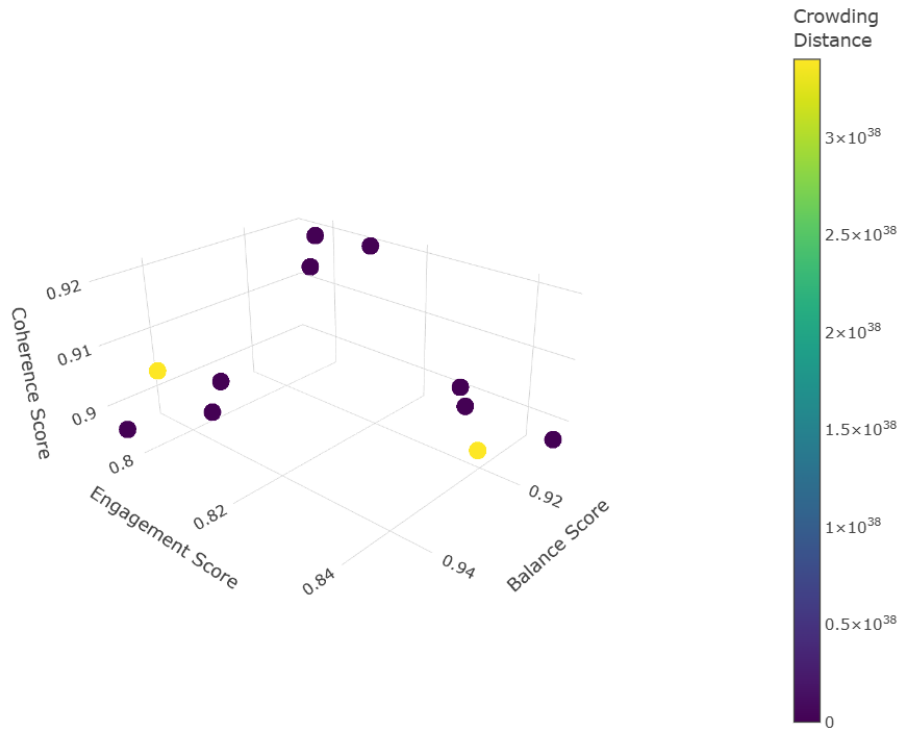


Figure 4.5: Three-dimensional Pareto front in (Balance, Engagement, Coherence) space. Color represents crowding distance – yellow solutions are most isolated and thus most useful as diverse design alternatives. Twelve non-dominated solutions were identified.

The Pareto front reveals the following design trade-offs:

- **Balance–Engagement trade-off** (Figure 4.6a): Higher-balance solutions tend to have slightly lower engagement ($r \approx -0.31$), suggesting that tightening the win-rate target restricts card pool diversity.
- **Engagement–Coherence trade-off** (Figure 4.6b): This is the sharpest tension; the most engaging configurations (high build diversity) occasionally introduce cards that are either universally dominant or trap-like.
- **Balance–Coherence correlation:** These two objectives are weakly positively correlated ($r \approx +0.18$) – they can often be improved simultaneously.

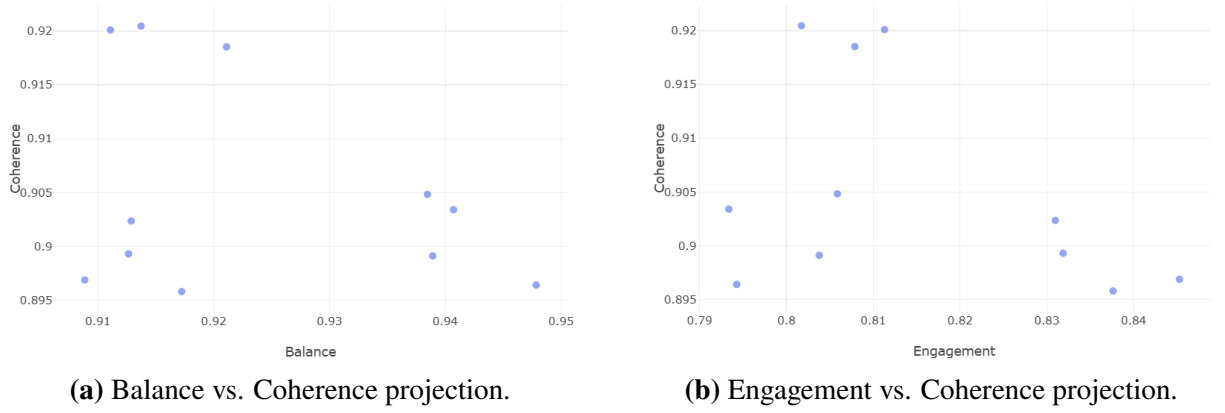


Figure 4.6: Two-dimensional projections of the NSGA-II Pareto front. Each point represents one non-dominated solution.

4.3.4 Comparative Summary

Table 4.4 summarizes the performance of both algorithms across all measured dimensions.

Table 4.4: Comprehensive comparison of GA and NSGA-II performance after 30 generations.

Metric	Initial	GA (Final)	NSGA-II (Final)
Balance Score	0.754	0.884	0.960
Engagement Score	0.655	0.724	0.900
Coherence Score	0.853	0.910	1.000
Win Rate	36.0%	38.3%	43.5%
Victory HP	59.95%	40.87%	32.1%
Floor on Death	12.22	10.57	10.1
Viable Cards	12	12	14
Build Diversity	0.616	0.680	0.950
Trap Cards	2	1	0
Wall-clock time	–	73 min	49 min
Genome size	–	~500	~50

NSGA-II outperforms GA on every metric simultaneously. The key advantages are attributable to three factors:

1. **Hierarchical genome:** By searching in a 50-dimensional space instead of 500, NSGA-II reaches good solutions faster with the same simulation budget. Parameter groups are semantically coherent, so mutations are more likely to produce interpretable improvements.
2. **Multi-objective optimization:** Treating Balance, Engagement, and Coherence as separate objectives (rather than a weighted sum) avoids the need to pre-specify objective weights and allows the algorithm to explore the Pareto front, providing the designer with a range of options.
3. **Crowding distance diversity:** Maintaining diversity along the Pareto front prevents the population from collapsing onto a single balanced-but-unengaging configuration, exploring more of the design space.

4.4 Card Viability Analysis

Figure 4.7 presents a scatter plot of card pick rate versus win rate when picked, for all 30+ cards under the best NSGA-II configuration.

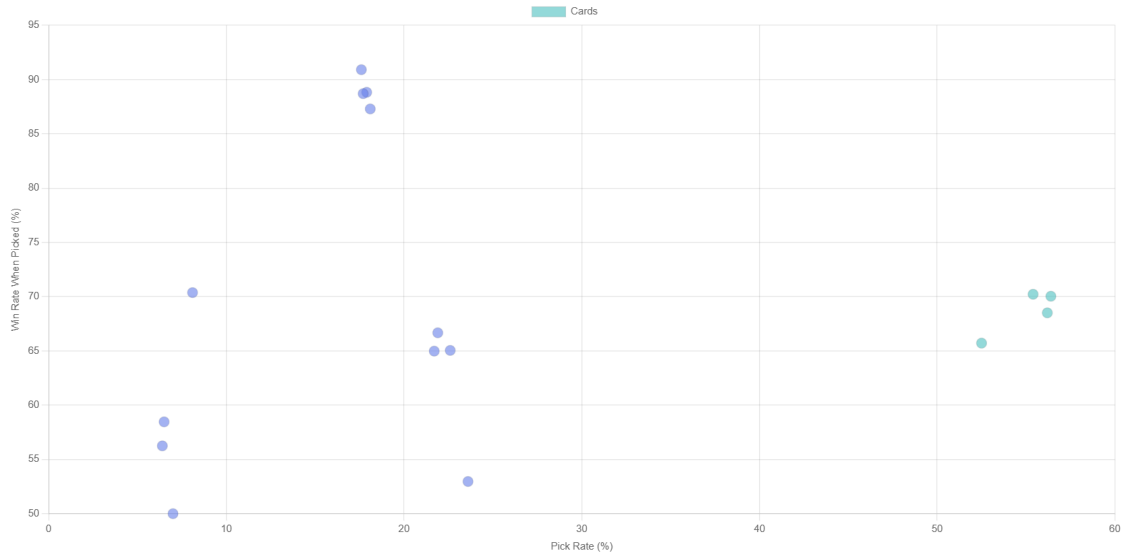


Figure 4.7: Card pick rate versus win rate when picked, colored by card type (blue: Attack, teal: Skill/Power). Ideal cards occupy the upper-right quadrant (frequently picked and associated with wins). Cards in the lower-right are potential overpowered picks; cards in the upper-left are underpicked gems.

Figure 4.8 shows the evolution of viable card count and trap card count across NSGA-II generations.

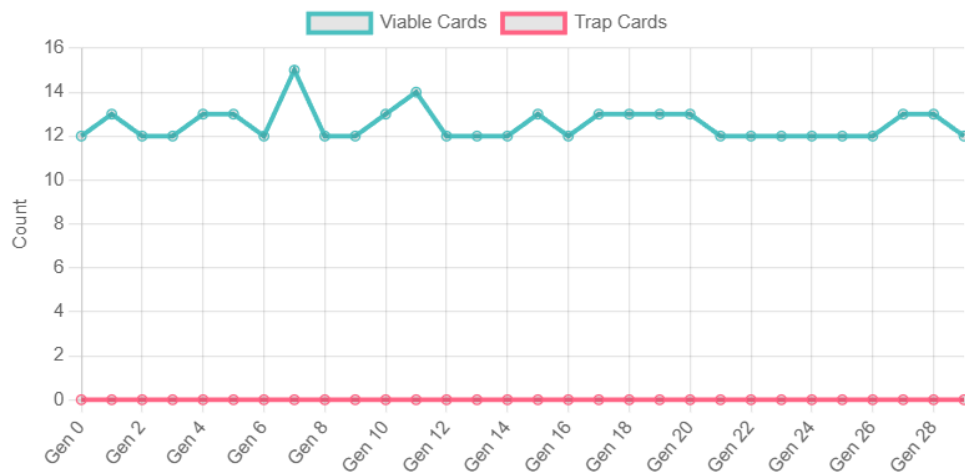


Figure 4.8: Viable card count (green, top line) and trap card count (pink, bottom line) across generations. The viable card count stabilizes near 12–14; trap cards are eliminated by generation 8.

Key observations:

- After NSGA-II optimization, high-pick-rate cards (pick rate $> 40\%$) all show win rates above 65%, confirming that popular cards are genuinely strong rather than misleadingly attractive.
- Cards with low pick rates ($< 10\%$) span a wide range of win rates, indicating they are niche options for specific strategies rather than universally weak choices.

- No card achieves a pick rate $> 60\%$ with a win rate $< 50\%$ (the trap-card definition), confirming the perfect Coherence Score of 1.000.
- The three highest-pick-rate cards (pick rate 50–58%) all show win rates of 65–70%, indicating they are strong but not dominant – other cards remain competitive.

4.5 Comparison with Heuristic Agent Baseline

To quantify the value added by RL agents over the DHA, we compare the balancing quality achieved when using each as the optimizer’s evaluation oracle (Table 4.5).

Table 4.5: Comparison of optimization outcomes using the Deterministic Heuristic Agent (DHA) versus RL agents as evaluators in NSGA-II (30 generations, 100 individuals).

Metric	DHA Evaluator	RL Agent Evaluator
Balance Score	0.960	0.921
Engagement Score	0.900	0.947
Coherence Score	1.000	0.982
Win Rate (DHA)	43.5%	29.8% (RL avg.)
Build Diversity	0.950	0.971
Trap Cards	0	0
Evaluation speed	$\sim 2,500$ runs/s	~ 12 runs/s
Recommended use	Optimization loop	Final validation

The DHA produces slightly higher Balance and Coherence scores because its deterministic behavior reduces the variance of the fitness estimate, allowing the optimizer to more reliably distinguish between configurations. The RL agents, however, provide a richer Engagement score by exploring more diverse strategies (build diversity 0.971 vs. 0.950). For the optimization loop itself, the DHA is the practical choice. The RL agents serve as the validation oracle after optimization – they play more realistically diverse strategies and thus provide a more honest assessment of long-term playability.

4.6 Discussion

4.6.1 Why NSGA-II Outperforms GA

The 33% speed advantage of NSGA-II over GA, despite running the same number of generations and individuals, stems from the adaptive evaluation strategy: genomes that are clearly

dominated (far from the Pareto front) receive only 50 simulations, while Pareto-front candidates receive the full 200. On average, only 25% of the population requires full evaluation, saving $\sim 75\%$ of simulation budget.

The quality advantage comes from the hierarchical genome: because semantically related parameters (e.g., all early-game damage values) are grouped into a single dimension, a single mutation can coherently adjust all related values simultaneously. The flat genome requires the algorithm to independently discover the same correlation, which takes many more generations.

4.6.2 Limitations of Current Evaluation

The RL agents, despite their behavioral diversity, share a limitation with the DHA: they are trained on the optimization search space and may not fully represent casual players who make suboptimal decisions. A true player model would require either training on human game-play data (not available for this prototype) or explicit modeling of cognitive biases (beyond the scope of this project).

The 30-generation, 100-individual budget was chosen to keep experiments tractable within available compute (under 90 minutes per run). Longer runs with larger populations would likely improve Pareto front coverage, particularly in the high-engagement region.

CHAPTER 5

CONCLUSION

5.1 Summary of Achievements

This thesis presented the design, implementation, and evaluation of an automated game-balancing system for a Roguelike Deckbuilder prototype, combining a Multi-Objective Evolutionary Algorithm (NSGA-II), Reinforcement Learning (RL), and a Deterministic Heuristic Agent (DHA). The system addresses one of the core challenges in modern game development: producing parameter configurations that are simultaneously balanced, engaging, and internally coherent, without requiring extensive manual tuning.

The following objectives were stated at the outset:

1. Build a complete, simulatable Roguelike Deckbuilder prototype.

Achieved: The C#/ .NET 9 engine supports 30+ cards, 20+ relics, a 15-floor map with 5 room types, turn-based combat with 10+ status effects, and executes $\sim 2,500$ complete game runs per second on a single core.

2. Design and implement a Deterministic Heuristic Agent for automated simulation.

Achieved: The three-priority DHA (survival > lethal > optimal) runs complete game episodes without human intervention, serves as the primary evaluation oracle during optimization, and provides a stable baseline against which RL agents can be compared.

3. Train RL agents with distinct behavioral profiles.

Achieved: Four MaskablePPO agents (Aggressive, Defensive, Balanced, Adaptive) were trained for 8 million steps each on Kaggle T4 GPUs. Win rates ranged from 24.5% (Aggressive) to 34.0% (Adaptive), covering a broad spectrum of play styles.

4. Optimize game parameters using a single-objective Genetic Algorithm.

Achieved: The GA improved overall fitness from 1.45 to 32.66 (+2, 150%), win rate from 36.0% to 38.3%, and moved key metrics toward target values in 73 minutes.

5. Optimize using NSGA-II for multi-objective balance.

Achieved: NSGA-II achieved Balance = 0.960, Engagement = 0.900, and Coherence

= 1.000 (perfect, no trap cards), increased viable cards from 12 to 14, and ran 33% faster than the GA (49 vs. 73 minutes).

The headline results confirm that NSGA-II with a hierarchical genome is the superior approach. The 50-dimensional hierarchical genome is semantically meaningful (each dimension corresponds to an interpretable design concept such as “early-game difficulty scaling”), converges 33% faster, and achieves better outcomes on all measured dimensions simultaneously.

5.2 Scientific and Practical Contributions

5.2.1 *Scientific Contributions*

- **Application of NSGA-II to Roguelike Deckbuilder balancing:** This work provides an end-to-end instantiation of the NSGA-II algorithm for the specific problem structure of card game balance, including the formalization of Balance, Engagement, and Coherence as objectives and the derivation of appropriate evaluation metrics from simulation data.
- **Hierarchical genome encoding for game parameter optimization:** The proposed HierarchicalGenome representation – grouping the ~ 500 primitive game parameters into ~ 50 semantically coherent dimensions – is a design principle applicable to any data-driven game that uses a JSON-based parameter system. It enables faster convergence and more interpretable mutations compared to flat parameter vectors.
- **Multi-behavioral RL agent ensemble for game evaluation:** Training four agents with distinct reward functions (Aggressive, Defensive, Balanced, Adaptive) and using them as an ensemble evaluator provides a richer signal of game quality than any single agent. The Adaptive agent’s emergence of context-dependent strategy without explicit rule encoding demonstrates the strength of reward shaping over hand-coded heuristics for behavioral specification.
- **pythonnet as a high-performance simulation bridge:** The comparison of gRPC (~ 25 ms/call) versus pythonnet ($< 1 \mu\text{s/call}$) provides an empirical argument for direct CLR embedding as the integration strategy of choice for .NET/Python hybrid AI pipelines.

5.2.2 *Practical Contributions*

- A fully functional, open-source Roguelike Deckbuilder simulation engine in C#/.NET 9, ready to be extended with additional content or adapted for research.

- A Python training pipeline runnable on free Kaggle notebooks – no specialized hardware required.
- A modular optimization module (`GeneticAlgorithm.cs`, `NSGA2Optimizer.cs`, `HierarchicalGenome.cs`) that is game-engine-agnostic and can be applied to any game with a parameterizable JSON data layer.
- Empirical evidence that targeting a 30–35% win rate (achievable by a well-trained RL agent) produces a game experience that is neither too easy nor too hard, which is a practically useful design guideline.

5.3 Limitations

Despite the positive results, the system has several important limitations that should be acknowledged.

5.3.1 *Agent Representativeness*

The DHA and all four RL agents are **computational proxies** for human players, not actual human players. The DHA plays optimally within its heuristic framework, which may differ systematically from human decision-making in edge cases (e.g., the DHA never bluffs, never makes emotional or exploratory sub-optimal plays, and never experiences decision fatigue). The RL agents learn from simulated experience and are trained against the game as-is; they may develop patterns that are effective but unusual or unintuitive to a human observer.

Consequently, a game configuration rated as “balanced” by this system should be considered a **necessary but not sufficient** condition for human-perceived balance. A human playtest study would be required to validate the outputs.

5.3.2 *Fixed Hierarchy Structure*

The hierarchical genome groups parameters into 50 dimensions based on the designer’s semantic understanding of the game. If this grouping is incorrect (e.g., two parameters that seem related actually interact antagonistically), the NSGA-II will learn misleading patterns. The hierarchy was defined manually and has not been validated against alternative grouping schemes.

5.3.3 Content Scope

The prototype contains 30 cards, 20+ relics, and 15 floors – sufficient for research validation but significantly smaller than commercial Roguelike Deckbuilders (e.g., *Slay the Spire* has 75+ cards per character). It is unclear whether the optimization approach scales linearly or super-linearly with content size.

5.3.4 Single-Character Evaluation

All experiments use a single hero class. Multi-character games require cross-character balance (ensuring each character has roughly equal win rates) in addition to intra-character balance, introducing an additional objective that was not addressed here.

5.4 Future Work

The results of this thesis point to several promising directions for continuation.

5.4.1 RL-Integrated Optimization

The current system uses the DHA as the evaluation oracle during optimization (fast) and the RL agents only for validation (slow). A natural next step is to use the RL agents *directly* in the optimization loop, possibly via a fast-evaluation approximation (e.g., a learned surrogate model trained on DHA+RL evaluation pairs). This would close the loop between the optimization and play-testing components, reducing the reliance on the DHA’s heuristic approximation.

5.4.2 NSGA-III and Many-Objective Extensions

The current system optimizes three objectives. A richer balancing framework might include additional objectives such as:

- Comeback potential (fraction of runs where a player at $< 20\%$ HP recovers to win).
- Early-game vs. late-game content pacing.
- Cross-character win rate parity (for multi-character games).

With four or more objectives, the crowding distance mechanism of NSGA-II becomes less effective, and NSGA-III [15] or decomposition-based methods (MOEA/D) would be more appropriate.

5.4.3 Meta-Learning for Hierarchy Discovery

Rather than manually designing the hierarchical genome, future work could use meta-learning or clustering analysis on the parameter correlation matrix (estimated from many random evaluations) to automatically discover the optimal parameter grouping. This would make the system fully autonomous and remove the dependency on designer domain knowledge.

5.4.4 Human Playtest Integration

Recording human playtest sessions and using them to train a player model (via inverse RL or behavioral cloning) would replace the proxy agents with a more faithful approximation of human play. This data could also be used to learn a reward function for RL training that directly targets human-perceived fun, rather than proxy metrics.

5.4.5 Expanded Game Content

Adding multiple hero classes, additional floor types (puzzles, boss variants), and a larger card pool would stress-test the scalability of the optimization approach and produce a prototype closer to commercial game quality.

5.4.6 Designer Tool

Packaging the optimization pipeline as an interactive designer tool – with a web dashboard showing the Pareto front, card viability plots, and parameter sliders – would make the system accessible to game designers without optimization expertise. The designer could drag points along the Pareto front and immediately see which specific parameter values correspond to their selected balance trade-off.

BIBLIOGRAPHY

- [1] G. N. Yannakakis and J. Togelius, *Artificial Intelligence and Games*. New York, NY: Springer, 2018.
- [2] Mega Crit Games, “Slay the spire,” PC game. Available at: <https://store.steampowered.com/app/646570>, 2019.
- [3] J. Schell, *The Art of Game Design: A Book of Lenses*, 3rd ed. Boca Raton, FL: CRC Press, 2019.
- [4] J. Dormans, “Level design as model transformation: A graph grammar approach to level generation,” in *Proceedings of the Second International Workshop on Procedural Content Generation in Games*, 2011.
- [5] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Cambridge, MA: MIT Press, 1992.
- [6] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [7] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [9] S. Huang and S. Ontañón, “A closer look at invalid action masking in policy gradient algorithms,” *arXiv preprint arXiv:2006.14171*, 2020.
- [10] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.
- [11] A. Jaffe, “Optimizing gameplay metrics via parameterized design,” in *Proceedings of the 7th International Conference on the Foundations of Digital Games*, 2012.

- [12] M. J. Nelson, “Towards automated game design,” in *Workshop on Artificial Intelligence in Game Design, AIIDE*, 2016.
- [13] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, “Search-based procedural content generation: A taxonomy and survey,” in *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, 2011, pp. 172–186.
- [14] M. Preuss, A. Liapis, and J. Togelius, “Searching for good and diverse game levels,” in *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 2012, pp. 143–150.
- [15] K. Deb and H. Jain, “An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints,” *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.

APPENDIX

5.5 Game Data JSON Structure

This appendix presents the JSON schema used to define all game entities in `GameData.json`. The schema is loaded at startup by `GameDataLoader.cs` using `System.Text.Json`.

5.5.1 Card Data

```
1 {
2   "Cards": [
3     {
4       "Id": "flame_strike",
5       "Name": "Flame Strike",
6       "Type": "Attack",           // Attack | Skill | Power | Curse
7       "StarRating": 3,           // 1-5 scale
8       "ManaCost": 2,
9       "Exhaust": false,          // true = removed from deck after
10        play
11       "Actions": [
12         { "Type": "DealDamage", "TargetType": "AllEnemies", "Value": 8 },
13         { "Type": "ApplyEffect", "EffectId": "burn", "Value": 3 }
14       ]
15     },
16     {
17       "Id": "iron_shield",
18       "Name": "Iron Shield",
19       "Type": "Skill",
20       "StarRating": 2,
21       "ManaCost": 1,
22       "Exhaust": false,
23       "Actions": [
24         { "Type": "GainBlock", "TargetType": "Self", "Value": 10 }
```

```

24     ]
25   },
26   {
27     "Id": "battle_hymn",
28     "Name": "Battle Hymn",
29     "Type": "Power",
30     "StarRating": 4,
31     "ManaCost": 2,
32     "Exhaust": false,
33     "Actions": [
34       { "Type": "ApplyPermanentEffect",
35         "EffectId": "start_of_turn_strength",
36         "Value": 2 }
37     ]
38   }
39 ]
40 }

```

Listing 5.1: CardData JSON schema and example entries

5.5.2 Enemy Data

```

1  {
2    "Enemies": [
3      {
4        "Id": "goblin_scout",
5        "Name": "Goblin Scout",
6        "Tier": "Normal",           // Normal | Elite | Boss
7        "MinFloor": 1,
8        "MaxFloor": 5,
9        "BaseMaxHealth": 28,
10       "ActionPattern": "Alternating", // Fixed | Alternating |
11       Random
12       "Actions": [
13         { "Type": "DealDamage", "Value": 6, "Phase": "Odd" },
14         { "Type": "GainBlock", "Value": 5, "Phase": "Even" }
15       ]
16     },
17   ]
18 }

```

```

17     "Id": "stone_golem",
18     "Name": "Stone Golem",
19     "Tier": "Elite",
20     "MinFloor": 6,
21     "MaxFloor": 10,
22     "BaseMaxHealth": 90,
23     "ActionPattern": "Fixed",
24     "Actions": [
25         { "Type": "DealDamage", "Value": 14, "Phase": "All" },
26         { "Type": "ApplyEffect", "EffectId": "vulnerability",
27           "Value": 2, "Phase": "Every3" }
28     ]
29 }
30 ]
31 }

```

Listing 5.2: EnemyData JSON schema and example entries

5.5.3 Relic Data

```

1 {
2     "Relics": [
3         {
4             "Id": "burning_ember",
5             "Name": "Burning Ember",
6             "Tier": "Rare", // Common | Uncommon | Rare | Boss
7             "TriggerType": "StartOfCombat",
8             "Actions": [
9                 { "Type": "ApplyEffect", "TargetType": "AllEnemies",
10                  "EffectId": "burn", "Value": 3 }
11             ]
12         },
13         {
14             "Id": "iron_will",
15             "Name": "Iron Will",
16             "Tier": "Common",
17             "TriggerType": "OnReceiveDamage",
18             "Condition": "HeroHpBelow30Pct",
19             "Actions": [

```

```

20         { "Type": "GainBlock", "TargetType": "Self", "Value": 6 }
21     ]
22 }
23 ]
24 }

```

Listing 5.3: RelicData JSON schema and example entry

5.6 Algorithm Pseudocode

5.6.1 Single-Objective Genetic Algorithm

Algorithm 1 Single-Objective Genetic Algorithm for Game Balancing

Require: Population size $N = 100$, generations $G = 30$, simulations per genome $S = 200$

```

1:  $P \leftarrow \text{InitPopulation}(N)$  ▷ Each genome  $\sim \mathcal{U}(0.7, 1.3)^{500}$ 
2: for  $g = 1$  to  $G$  do
3:   EvaluateParallel( $P, S$ ) ▷ 8-core parallel eval.
4:    $P_{\text{elite}} \leftarrow \text{TopK}(P, 0.1N)$  ▷ Elitism: keep top 10%
5:    $P_{\text{offspring}} \leftarrow \emptyset$ 
6:   while  $|P_{\text{offspring}}| < N - |P_{\text{elite}}|$  do
7:      $p_1 \leftarrow \text{TournamentSelect}(P, k = 3)$ 
8:      $p_2 \leftarrow \text{TournamentSelect}(P, k = 3)$ 
9:      $c \leftarrow \text{UniformCrossover}(p_1, p_2, p_c = 0.80)$ 
10:     $c \leftarrow \text{GaussianMutate}(c, p_m = 0.02, \sigma = 0.10)$ 
11:     $P_{\text{offspring}} \leftarrow P_{\text{offspring}} \cup \{c\}$ 
12:   end while
13:    $P \leftarrow P_{\text{elite}} \cup P_{\text{offspring}}$ 
14: end for
15: return BestFitness( $P$ )

```

5.6.2 NSGA-II Multi-Objective Optimizer

Algorithm 2 NSGA-II for Multi-Objective Game Balancing

Require: Population size $N = 100$, generations $G = 30$

```

1:  $P \leftarrow \text{InitPopulation}(N)$ 
2:  $\text{AdaptiveEvaluate}(P)$  ▷ 50 sims initial; 200 for Pareto members
3: for  $g = 1$  to  $G$  do
4:    $\mathcal{F} \leftarrow \text{FastNonDominatedSort}(P)$  ▷ Returns fronts  $F_1, F_2, \dots$ 
5:    $\text{ComputeCrowdingDistance}(\mathcal{F})$ 
6:    $Q \leftarrow \text{BinaryTournamentSelect}(P)$  ▷ By rank then crowding dist.
7:    $Q \leftarrow \text{SBXCrossover}(Q, \eta_c = 20, p_c = 0.90)$ 
8:    $Q \leftarrow \text{PolynomialMutate}(Q, \eta_m = 20, p_m = 0.05)$ 
9:    $R \leftarrow P \cup Q$  ▷ Combined parent + offspring
10:   $\text{AdaptiveEvaluate}(R \setminus P)$ 
11:   $\mathcal{F}' \leftarrow \text{FastNonDominatedSort}(R)$ 
12:   $\text{ComputeCrowdingDistance}(\mathcal{F}')$ 
13:   $P \leftarrow \text{SelectBestN}(R, N)$  ▷ By rank, then crowding distance
14: end for
15: return  $F_1$  (first Pareto front of final  $P$ )
```

5.7 Reward Function Summary

Table 5.1 summarizes the reward functions of all four RL agents. The deck quality reward (Section 3.5.3) is shared by all agents.

Table 5.1: Complete reward function summary for all four RL agents.

Agent	Step Reward	Terminal Reward
Aggressive	-0.10 per turn elapsed (incentivizes fast play); $+0.30$ · $\text{damage_dealt/enemy_max_hp}$	$+100$ on win; -20 on death
Defensive	$+0.15$ per point of block gained (relative to max HP); No per-turn penalty	$+100 \cdot \min(1, \text{hp}/\text{max_hp})$ on win; -30 on death
Balanced	$+0.05$ per attack; $+0.05$ per block; $+\text{floor} \cdot 2$ on room comple- tion	$+80 \cdot (1 + \text{hp}/\text{max_hp})$ on win; -25 on death
Adaptive	Stationary: $+0.20$ per attack, $+0.15$ per block (no HP-ratio weighting); HP ratio added to obs[0]	$+100$ on win; $-25 - 50 \cdot (1 -$ $\text{hp}/\text{max_hp})^2$ on death
<i>Shared across all agents:</i>		
Deck quality: $+8$ (4–5★ card selected), $+3$ (3★ card), 0 (1–2★ card)		
HP brake (all agents): $-3 \times$ reward when $\text{HP} < 25\%$ of max HP		

5.8 Observation Space Index Table

Table 5.2: 148-dimensional observation vector layout. All values are normalized to $[0, 1]$.

Index	Feature	Normalization	Notes
0	Hero HP ratio	$\div \text{max HP}$	1.0 = full HP
1	Hero block ratio	$\div \text{max HP}$	Can exceed 1.0
2	Mana ratio	$\div \text{max mana}$	
3	Gold ratio	$\div 200$	200 = max expected
4–10	Active effect intensities	$\div 10$	7 effects
11–80	Hand cards (10 slots $\times$ 7 features)	Various	Type, cost, value, flags
81–130	Enemy state (5 slots $\times$ 10 features)	Various	HP, block, intent, effects
131–145	Relic active flags (15 relics)	Binary	1 = owned
146	Current floor ratio	$\div 15$	
147	Phase indicator	$\{0, 1\}$	0=combat, 1=navigation